Berkeley MSSE
MOLECULAR SCIENCE & SOFTWARE ENGINEERING

Lecture 12:

Clustering and Classification - Unsupervised



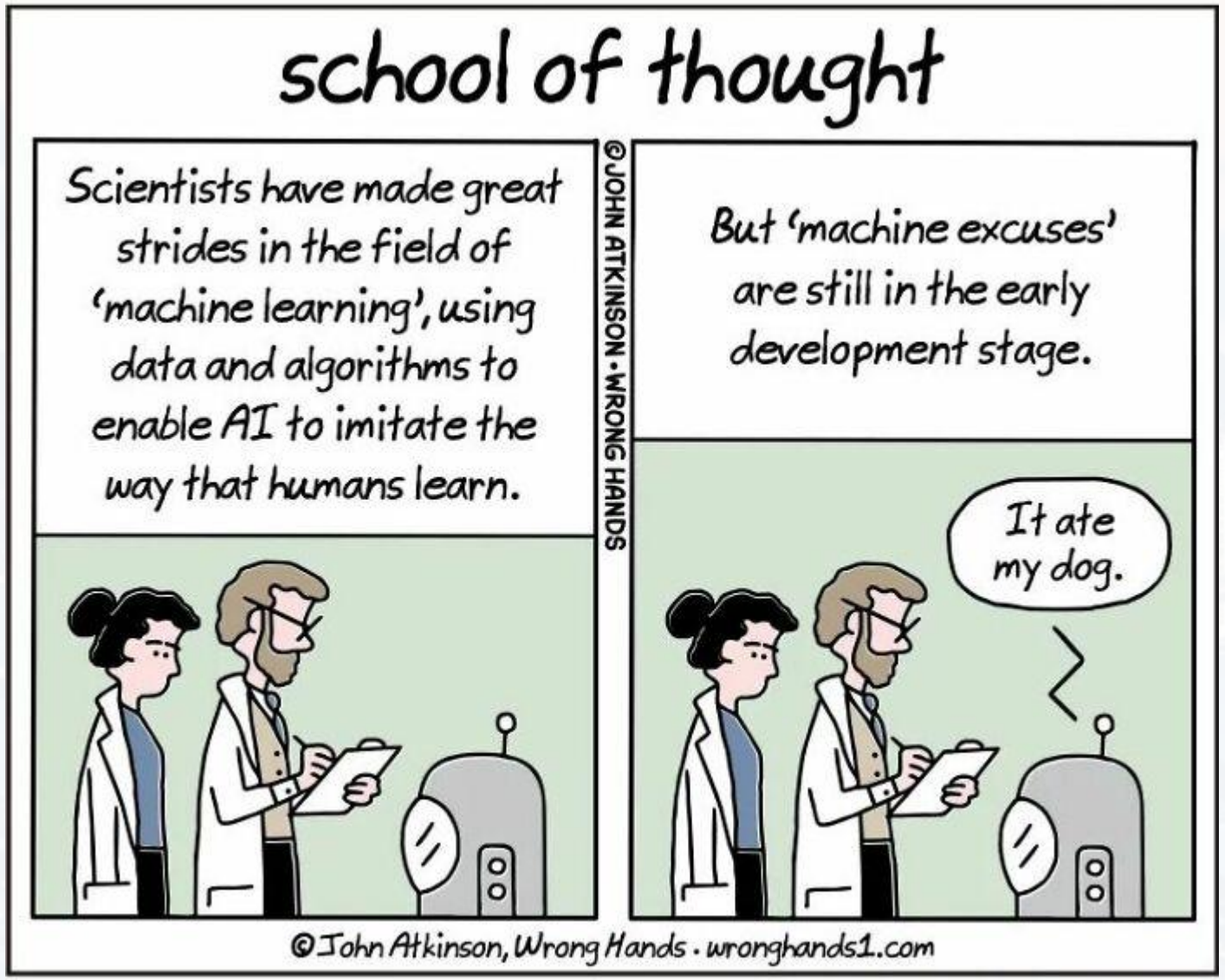
Markus Hohle

University of California, Berkeley

Data Science for Scientific Computing

MSSE 277A, 3 Units

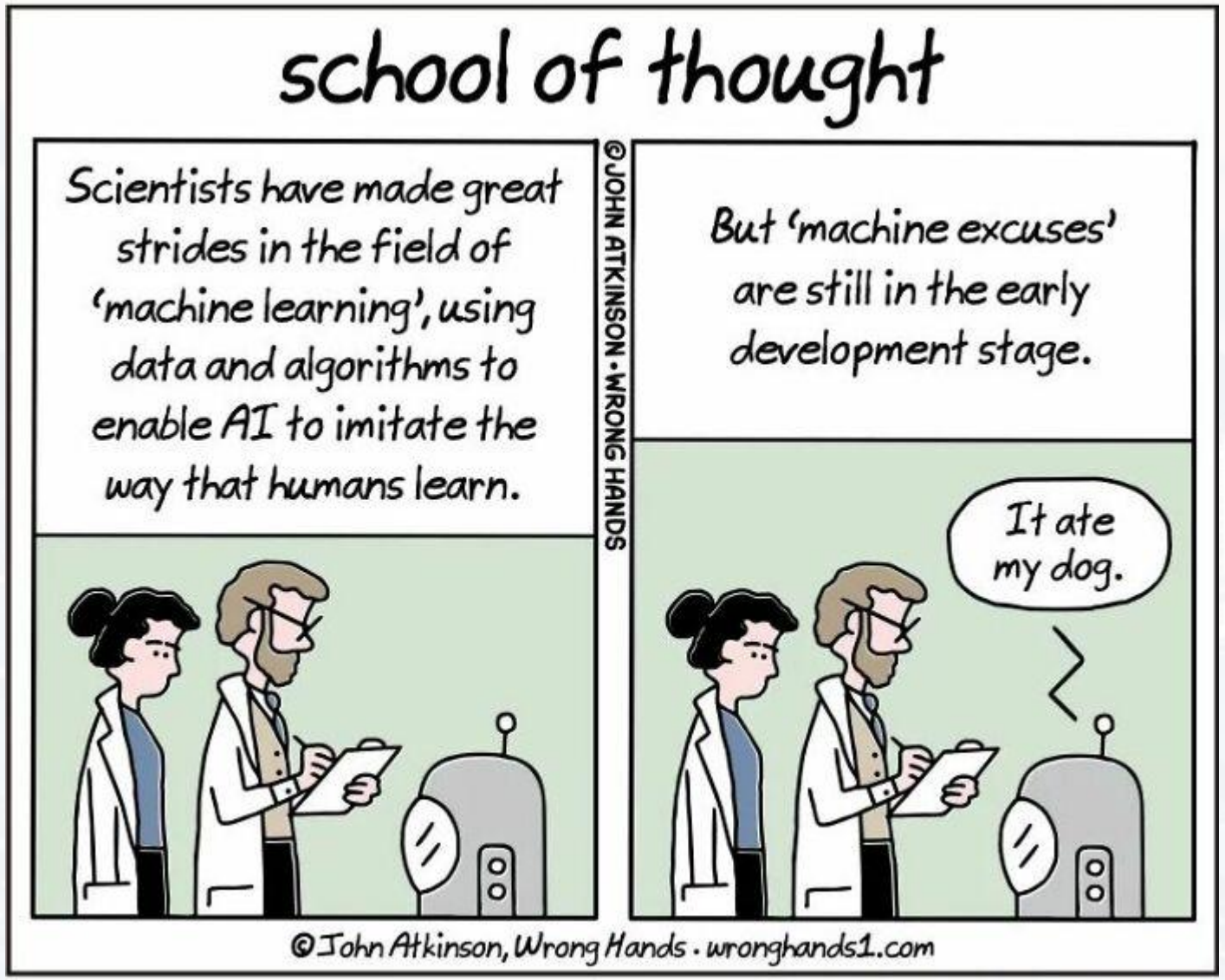
Clustering and Classification - Unsupervised



Outline

- **Overview**
- **More Statistics**
- **Examples**
  - k-means
  - GMMs
  - Trees

Clustering and Classification - Unsupervised



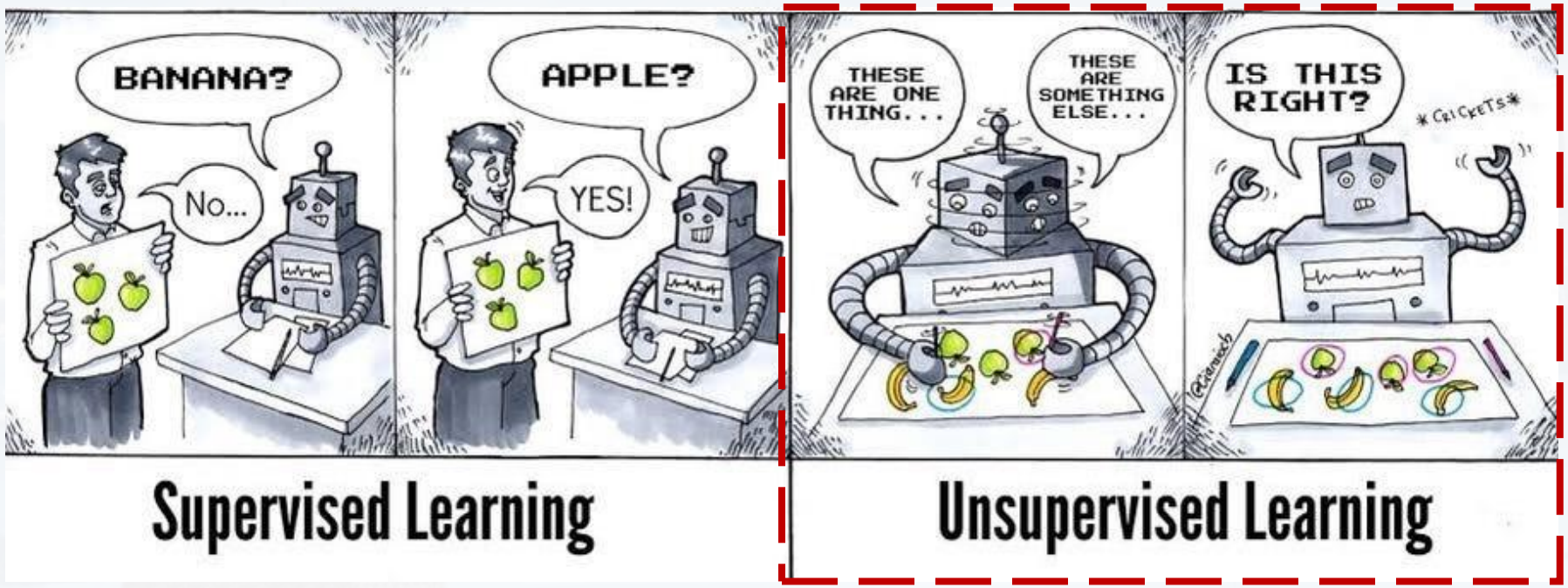
Outline

- **Overview**
- More Statistics
- Examples
  - k-means
  - GMMs
  - Trees

Clustering and Classification – Unsupervised

So far, there has been a **training** data set and a **test** data set…

… but maybe there are ways to learn *without* training data



Clustering and Classification – Unsupervised

	supervised	unsupervised
regression	linear regression ✓ ANNs (soon)	
classification	logistic regression ✓ K-nearest (soon) SVM (soon) ANNs (soon) Naïve Bayes (soon)	**GMM (today)** ✓ **K-means (today)** ✓ **Trees (today)** ✓ Hidden Markov Models

Workflow

supervised

1) creating the model:

```
my_model = library.method(argument1 = 'arg1', … )
```

2) training the model

```
out = my_model.fit(xtrain, ytrain)
```

3) evaluation

```
ypred = out.predict(xeval)
accur = (ypred == yeval).sum()/len(yeval)
```

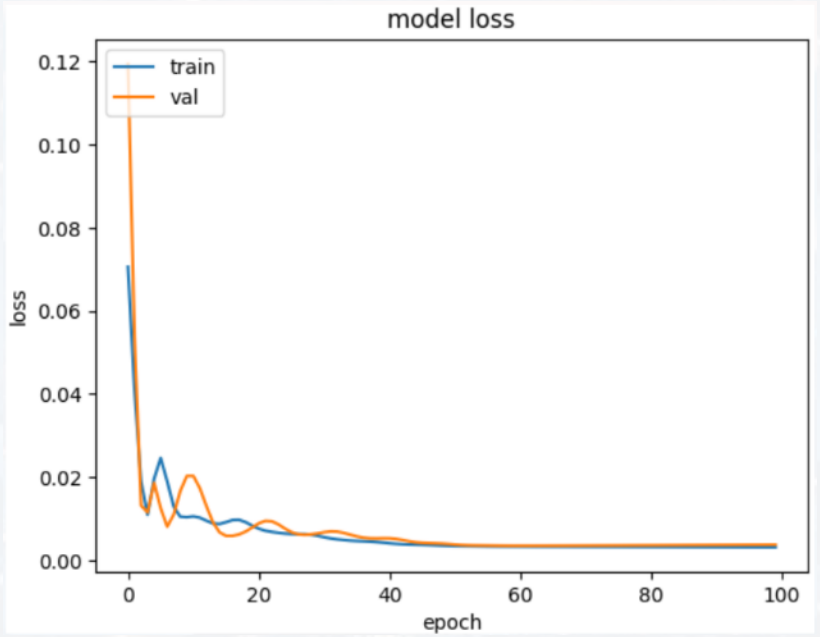
4) prediction (actual application)

```
ypred = out.predict(xnew)
```

How to split the data?

- 80% training **(60% actual training 20% evaluation)**
- 20% test

evaluation while training!



Clustering and Classification – Unsupervised

Workflow

supervised

1) creating the model:

```
my_model = library.method(argument1 = 'arg1', … )
```

2) training the model

```
out = my_model.fit(xtrain, ytrain)
```

3) evaluation

```
ypred = out.predict(xeval)
accur = (ypred == yeval).sum()/len(yeval)
```

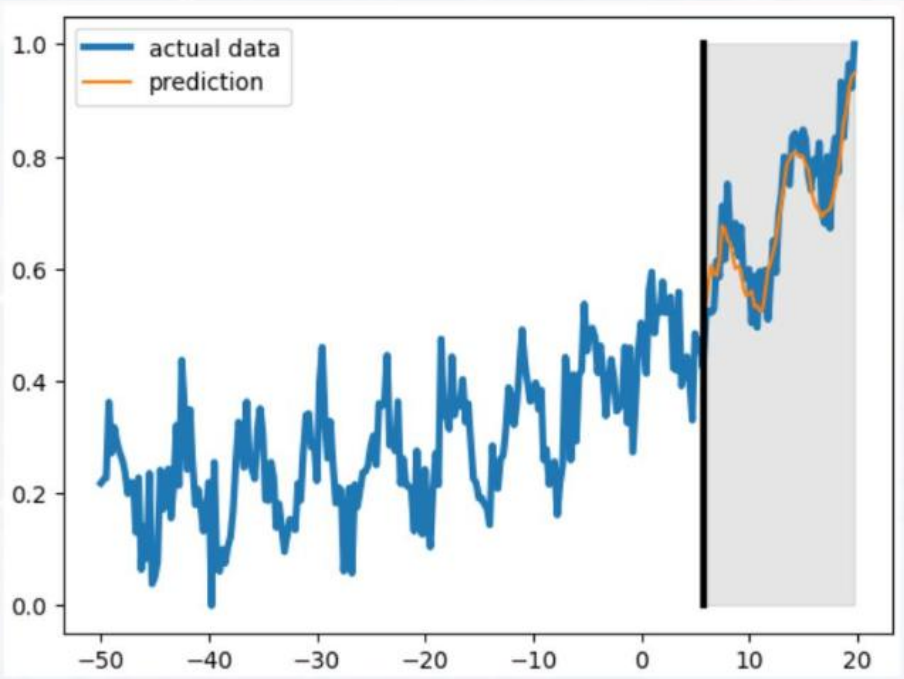
4) prediction (actual application)

```
ypred = out.predict(xnew)
```

How to split the data?

- 80% training **(60% actual training 20% evaluation)**
- 20% test

prediction after training/evaluation completed



Clustering and Classification – Unsupervised

unsupervised

Workflow

1) setting up the model

2) ~~fitting the model:~~ **X** are the **features** ~~(sepal lengths/widths and petal lengths/widths)~~,
Y are the **classes** (setosa, versicolor, virginica)

3) evaluating the model

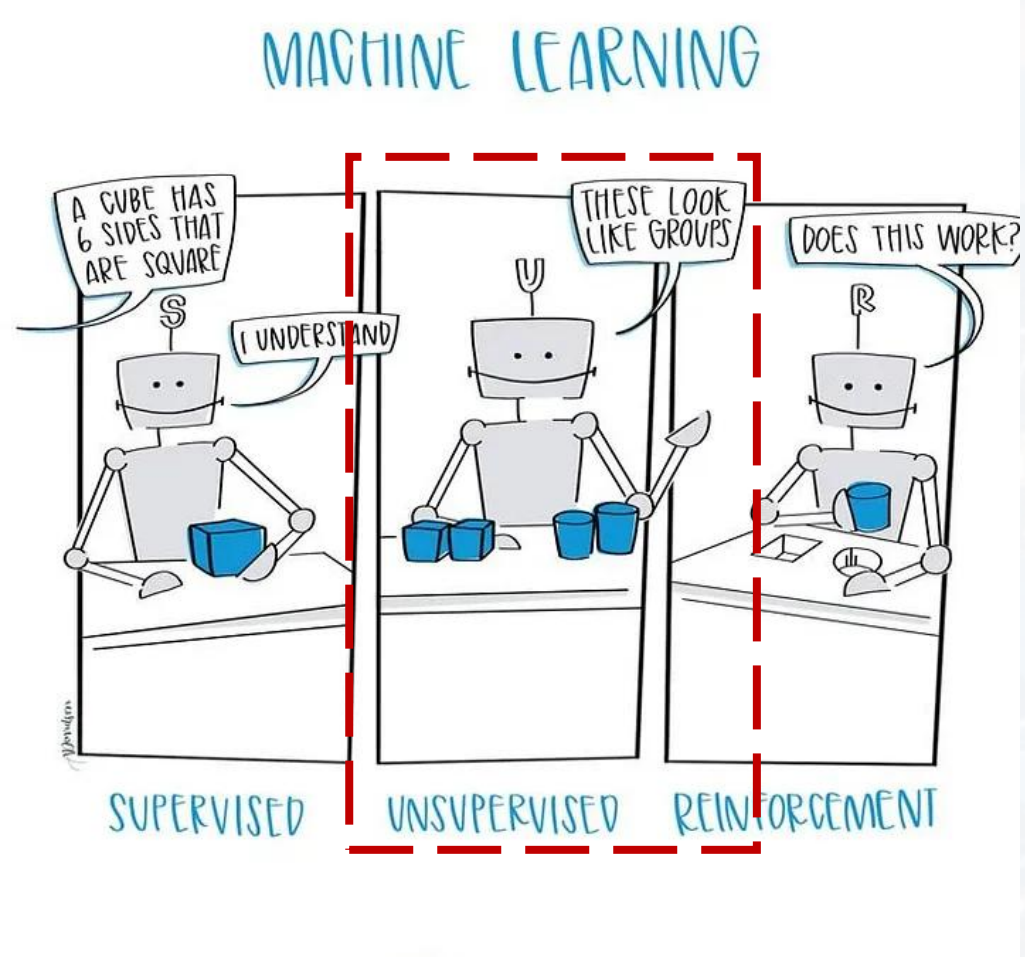
4) applying the model to a new data set

unsupervised learning:
- we don't know the classes
- sometimes we even don't know the **number *k* of classes**

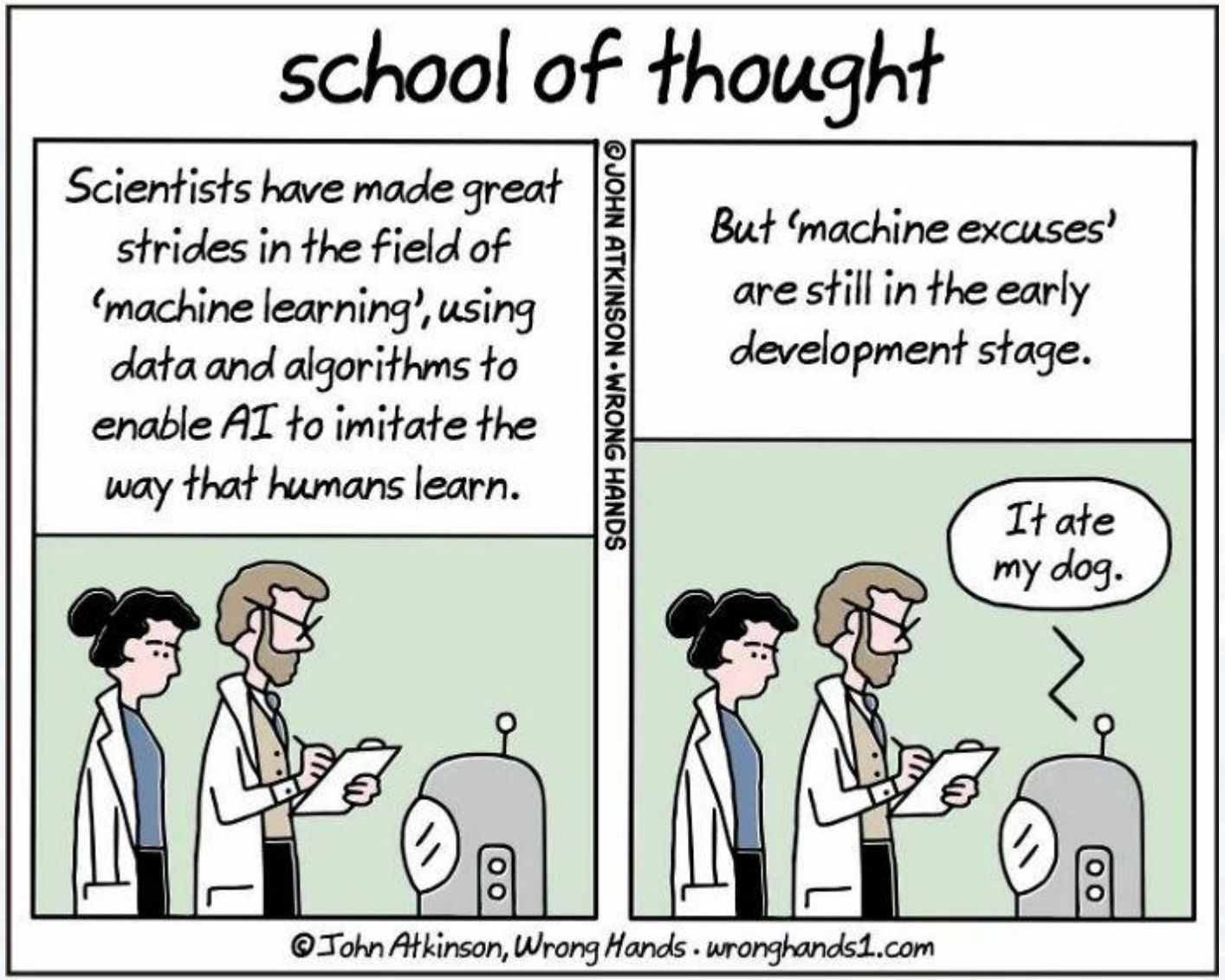
Clustering and Classification – Unsupervised

So far, there has been a **training** data set and a **test** data set…

… but maybe there are ways to learn *without* training data



Clustering and Classification - Unsupervised



Outline

- Overview
- **More Statistics**
- Examples
  - k-means
  - GMMs
  - Trees

Bayes Theorem

P(A ∩ B) probability ***P*** that the events ***A*** and ***B*** occur (intersection)

so far: A and B were independent $P(A \cap B) = P(A)P(B) = P(B)P(A)$

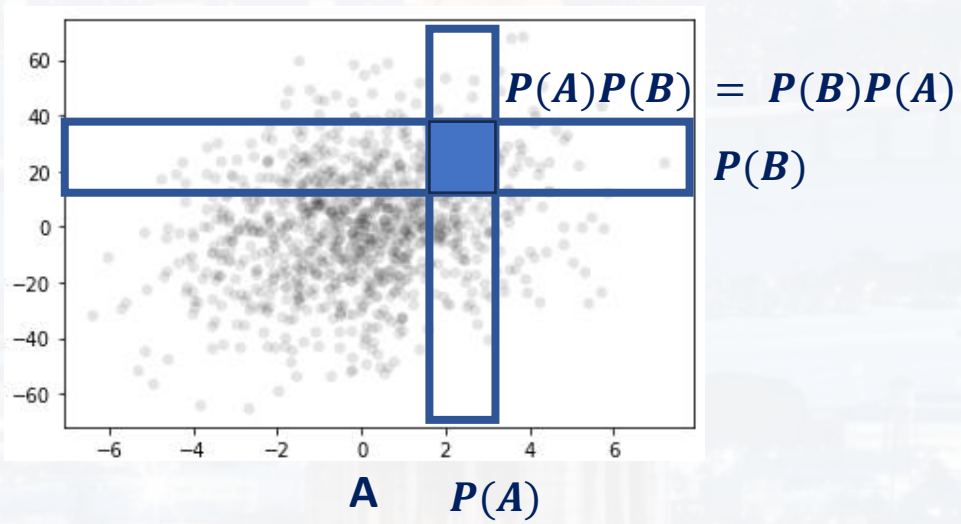
now: **conditional probabilities** | "given" or "under the condition"

$$P(\mathrm{A} \cap B) = P(A|B)P(B) = P(B|A)P(A)$$

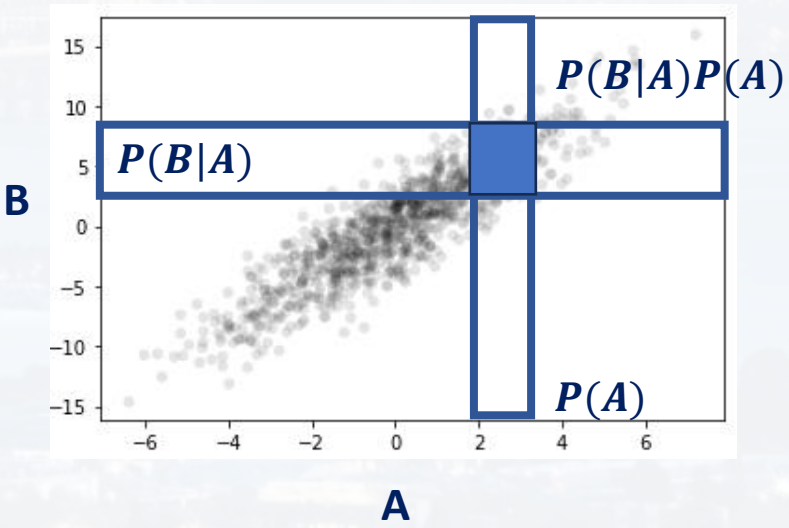


Thomas Bayes (1701 - 1761)

A and B are independent



A and B correlate!



Bayes Theorem P(A ∩ B) probability ***P*** that the events ***A*** and ***B*** occur (intersection)

so far: A and B were independent $P(A \cap B) = P(A)P(B) = P(B)P(A)$

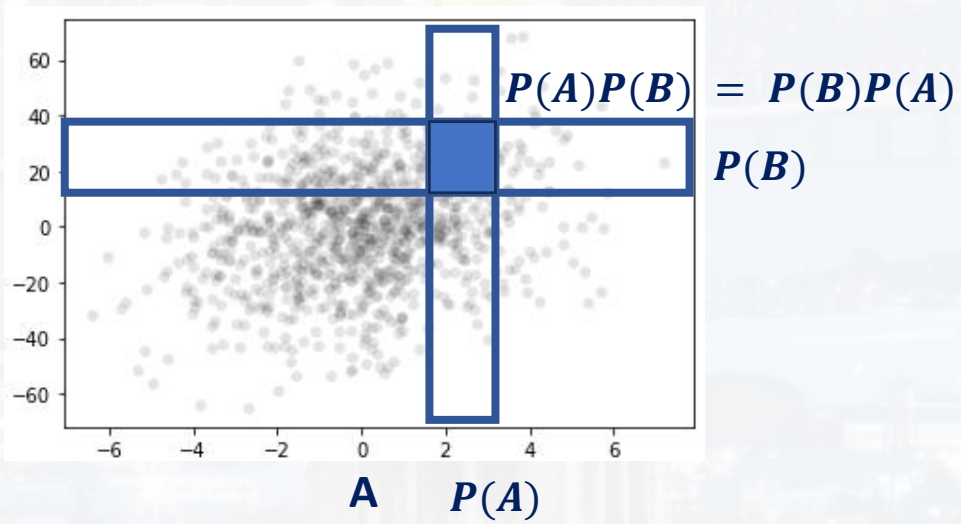
now: **conditional probabilities** | "given" or "under the condition"

$$P(\mathrm{A} \cap B) = P(A|B)P(B) = P(B|A)P(A)$$

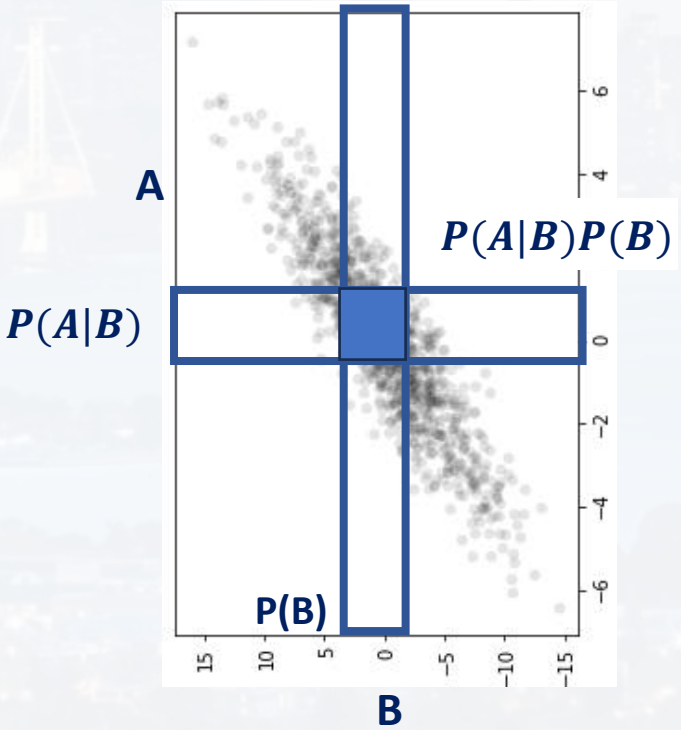


Thomas Bayes (1701 - 1761)

A and B are independent



A and B correlate!



Bayes Theorem

P(A ∩ B) probability ***P*** that the events ***A*** and ***B*** occur (intersection)



Thomas Bayes
(1701 - 1761)

so far: A and B were independent $P(A \cap B) = P(A)P(B) = P(B)P(A)$

now: **conditional probabilities** | "given" or "under the condition"

$P(\mathrm{A} \cap B) = P(A|B)P(B)$

→ $P(A|B)P(B) = P(B|A)P(A)$ **Bayes Theorem**

posterior $\boldsymbol{P(A|B)} = \dfrac{P(B|A)\boldsymbol{P(A)}}{P(B)}$ prior

Bayes Theorem

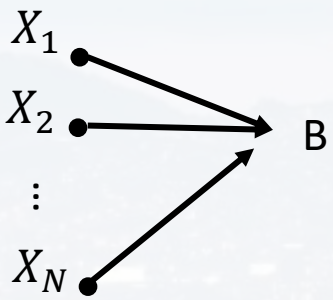
$$P(A|B)P(B) = P(B|A)P(A)$$

Bayes Theorem

posterior $$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$ prior



Thomas Bayes (1701 - 1761)



$$P(B) = \sum_{n=1}^{N} P(B|X_n)P(X_n)$$

$$P(B) = \int P(B|X)P(X)\,dX$$

marginalization

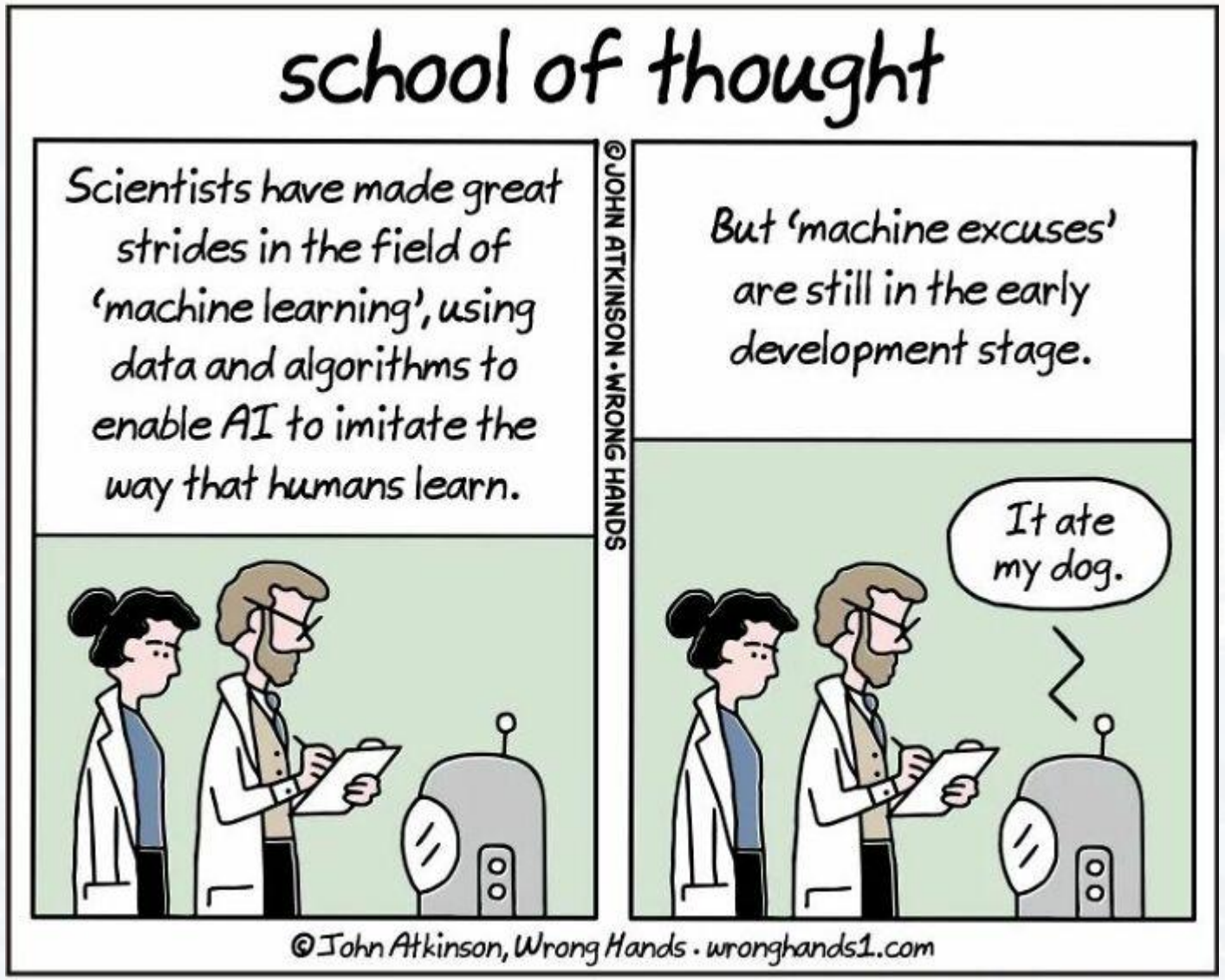
blood test: probability $P(+)$ to have a positive test result:

$$P(+) = P(+|sick)\,P(sick) + P(+|healthy)\,P(healthy)$$

p-value (module 11)

We will need these equations in the next chapter!

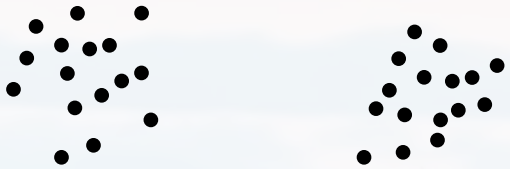
Clustering and Classification - Unsupervised



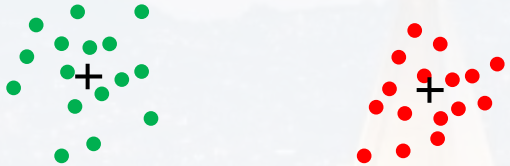
Outline

- Overview
- More Statistics
- **Examples**
  - k-means
  - GMMs
  - Trees

Clustering and Classification – Unsupervised



problem: we have N data points and don't know which data point belongs to which cluster
→ all data points x_n have the same color



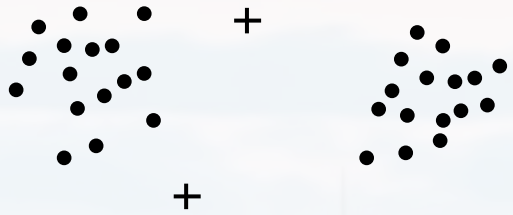
goal: assigning each data point to its cluster
→ classification (here: red vs green)

$$\mu_k = \frac{\sum_{n=1}^{N} r_{n,k} x_n}{\sum_{n=1}^{N} r_{n,k}}$$

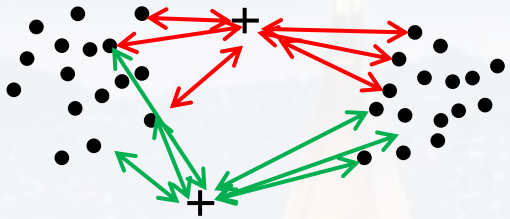
idea: assigning each data point x_n to its closest mean μ_k of cluster k

Clustering and Classification – Unsupervised

k – means is an iterative process



a) assign *k* means randomly



b) calculate *distance* from each point to each mean

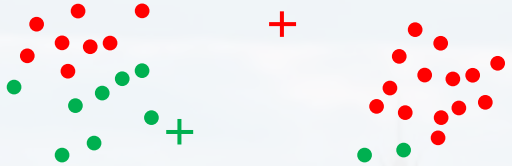


c) assign each point to its closest mean

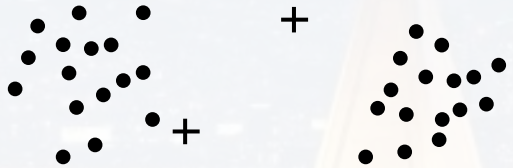


d) update the means accordingly

k – means is an iterative process



d) update the means accordingly



e) go back to b)



Clustering and Classification – Unsupervised

indicator function $r_{n,k} \in \{0, 1\}$

K	**: number of cluster**
N	**: number of observations**

goal: minimizing

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k} \ \|x_n - \mu_k\|^2$$

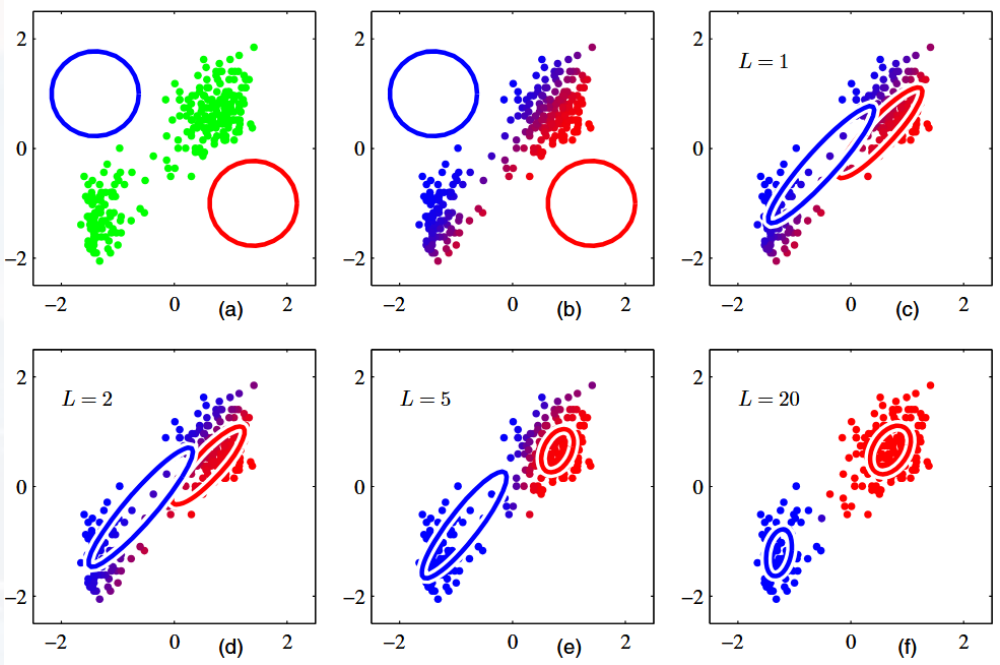
μ_k: means = barycenter of cluster *k*

assigning x_n to its closed mean

$$r_{n,k} = \begin{cases} 1, & if\ k = \ argmin_j \|x_n - \mu_j\|^2 \\ 0, & else \end{cases}$$

$$\frac{\partial J}{\partial \mu_k} = 0 \longrightarrow \mu_k = \frac{\sum_{n=1}^{N} r_{n,k} x_n}{\sum_{n=1}^{N} r_{n,k}}$$

k – means is an iterative process

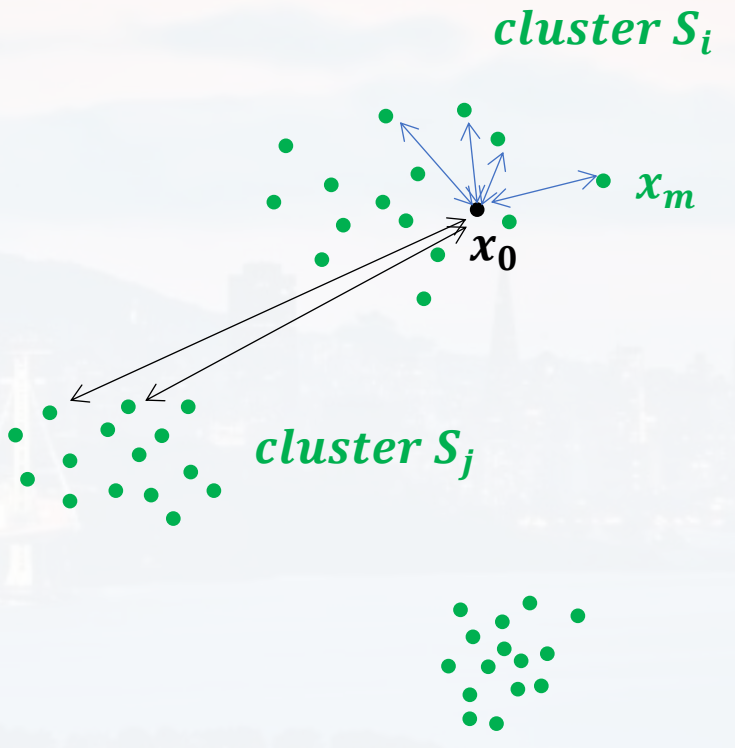


problem: ***k = number of cluster,*** is a hyperparameter. How do I know the correct value for ***k***?

→ silhouette Ψ

- distance $\boldsymbol{d_1}$ of a data point x_0 to ***its assigned cluster*** $\boldsymbol{S_i}$
  vs distance $\boldsymbol{d_2}$ to ***closest cluster (here*** $\boldsymbol{S_j}$***)***

$$\Psi(x_0) = \begin{cases} 0 & if\ d_1 = 0 \\ \dfrac{d_2 - \boldsymbol{d_1}}{max[\boldsymbol{d_1}; d_2]} & \end{cases}$$



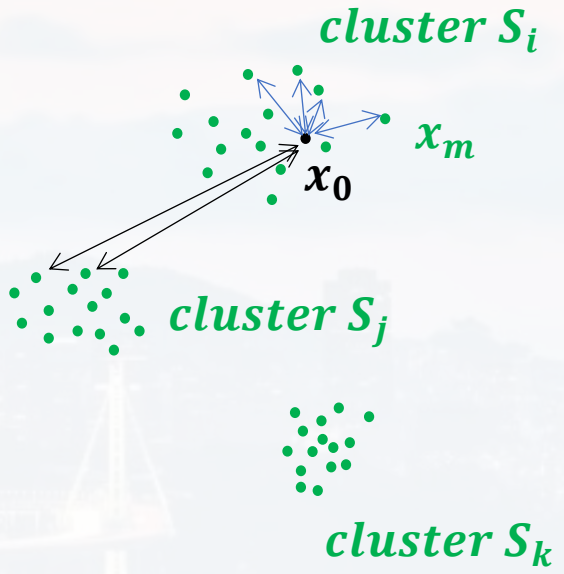
- average over all points → ψ_{tot}

if	$\psi_{tot} = 0.75 \ldots 1.00$	→ well clustered
	$\psi_{tot} = 0.50 \ldots 0.75$	→ medium clustered
	$\psi_{tot} = 0.25 \ldots 0.50$	→ poorly clustered
	$\psi_{tot} < 0.25$	→ data has no structure

problem: ***k = number of cluster,*** is a hyperparameter. How do I know the correct value for ***k***?

→ silhouette Ψ

- distance $\boldsymbol{d_1}$ of a data point x_0 to ***its assigned cluster*** $\boldsymbol{S_i}$
  vs distance $\boldsymbol{d_2}$ to ***closest cluster (here*** $\boldsymbol{S_j}$***)***



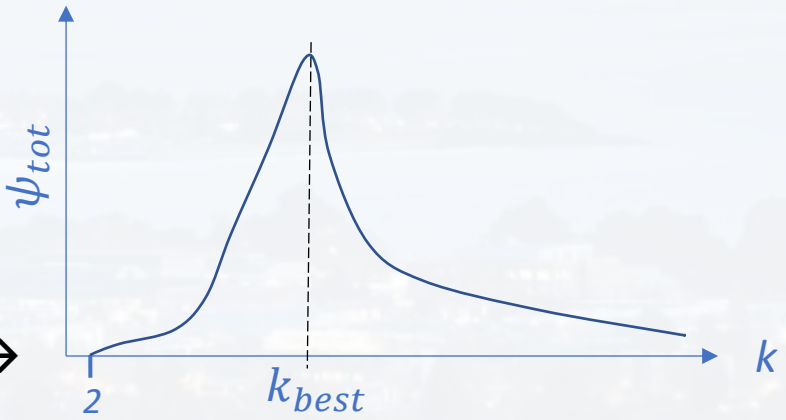
$$\Psi(x_0) = \begin{cases} 0 & if\ d_1 = 0 \\ \dfrac{d_2 - \boldsymbol{d_1}}{max[\boldsymbol{d_1}; d_2]} & \end{cases}$$

- average over all points → ψ_{tot}

if		
	$\psi_{tot} = 0.75 \ldots 1.00$	→ well clustered
	$\psi_{tot} = 0.50 \ldots 0.75$	→ medium clustered
	$\psi_{tot} = 0.25 \ldots 0.50$	→ poorly clustered
	$\psi_{tot} < 0.25$	→ data has no structure

see `Walk_Through_Kmeans.ipynb`

ideal world →



Clustering and Classification – Unsupervised


```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

our standard libraries

```
from pyclustering.utils.metric import *
```

for having different distances available

```
from nltk.cluster.kmeans import KMeansClusterer
```

```
from sklearn.metrics import silhouette_samples, silhouette_score
```

```
from sklearn import datasets
```

calling the famous “iris” data set

calculating silhouette coefficient for different k

performing k-means

see `Walk_Through_Kmeans.ipynb`

Clustering and Classification – Unsupervised


```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns

from pyclustering.utils.metric import *
from nltk.cluster.kmeans import KMeans(
from sklearn.metrics import silhouette_
from sklearn import datasets

iris = datasets.load_iris()

iris.DESCR
```

```
Iris plants dataset
--------------------

**Data Set Characteristics:**

    :Number of Instances: 150 (50 in each of three classes)
    :Number of Attributes: 4 numeric, predictive attributes and the class
    :Attribute Information:
        - sepal length in cm
        - sepal width in cm
        - petal length in cm
        - petal width in cm
        - class:
                - Iris-Setosa
                - Iris-Versicolour
                - Iris-Virginica

    :Summary Statistics:

    ============== ==== ==== ======= ===== ====================
                    Min  Max   Mean    SD   Class Correlation
    ============== ==== ==== ======= ===== ====================
    sepal length:   4.3  7.9   5.84   0.83    0.7826
    sepal width:    2.0  4.4   3.05   0.43   -0.4194
    petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
    petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
    ============== ==== ==== ======= ===== ====================
```

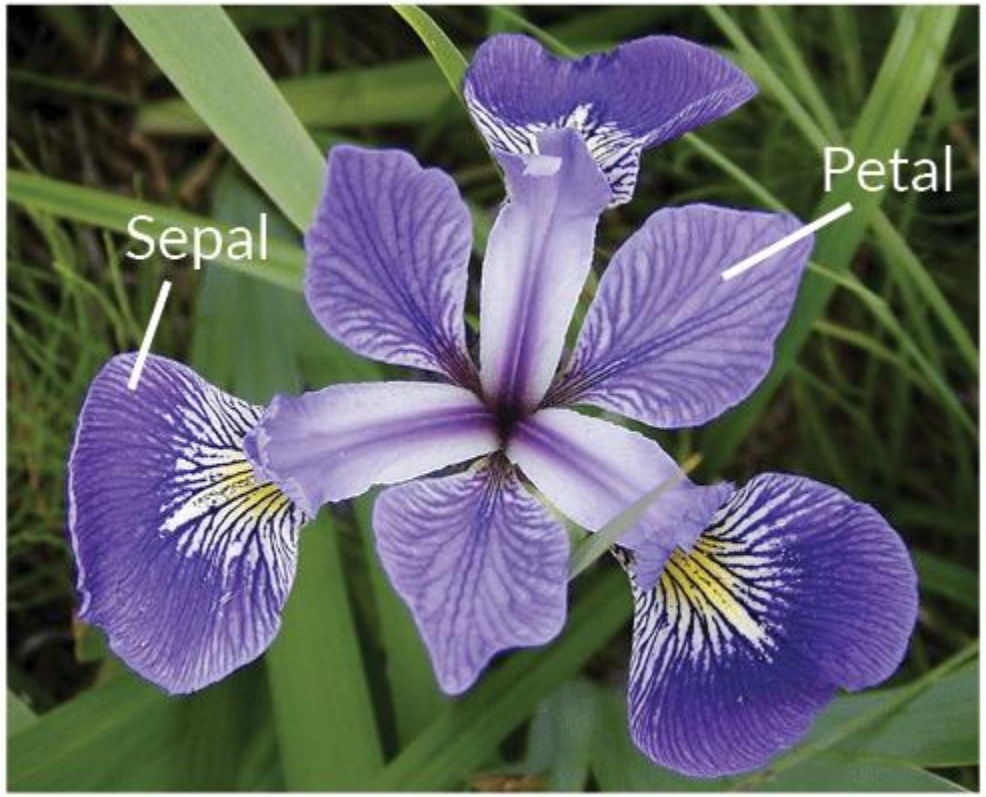
ideal world: three distinct cluster

see `Walk_Through_Kmeans.ipynb`

Clustering and Classification – Unsupervised


```
iris = datasets.load_iris()

iris.DESCR
```



Iris Versicolor



Iris Setosa



Iris Virginica

see `Walk_Through_Kmeans.ipynb`

Clustering and Classification – Unsupervised

loading & exploring the data:

```
iris = datasets.load_iris()

iris.DESCR
iris.feature_names
iris.target_names
```

```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

four features → 4D

```
array(['setosa', 'versicolor', 'virginica']
```

- plotting the data
- running k-means
- evaluating the result

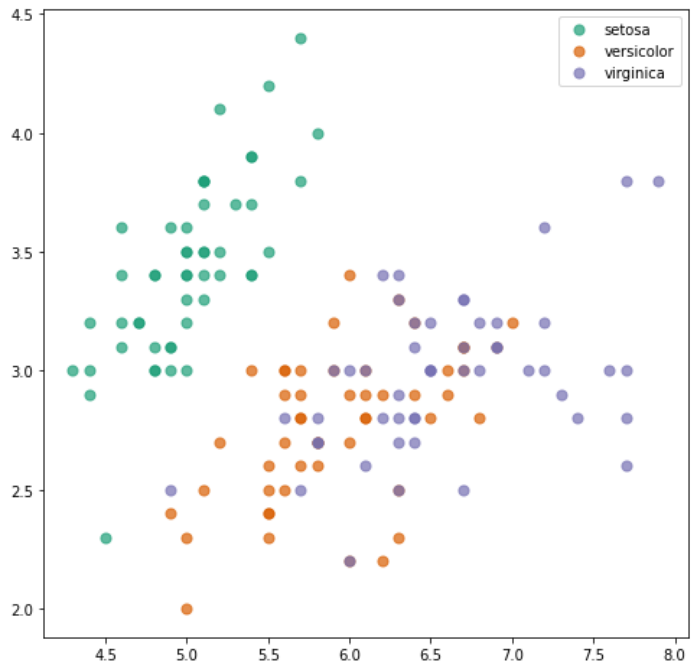
see `Walk_Through_Kmeans.ipynb`

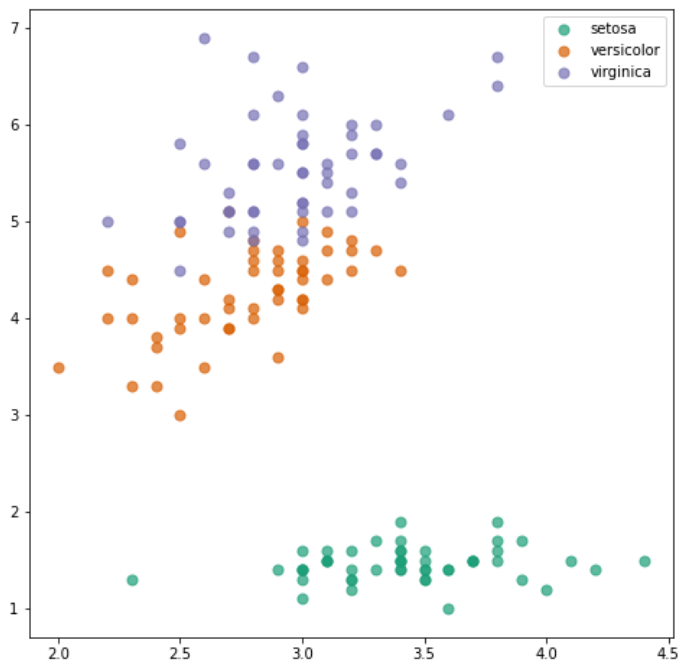
Clustering and Classification – Unsupervised

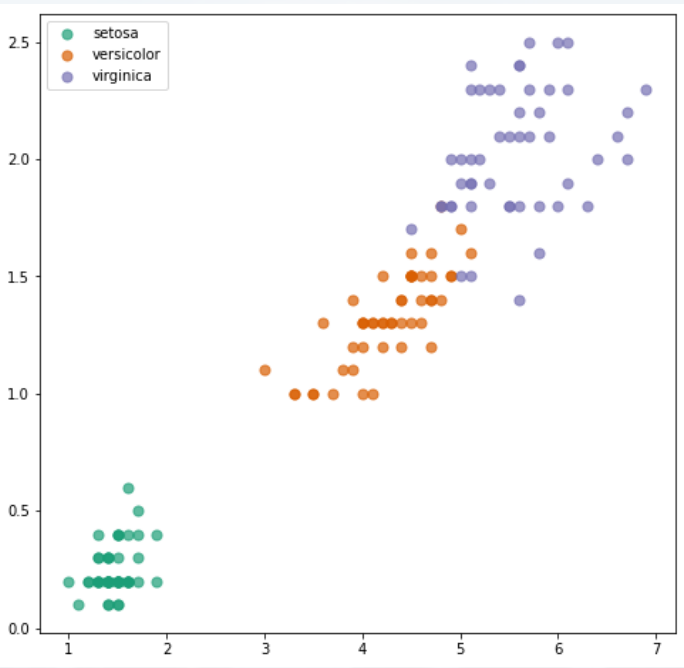

```
['sepal length (cm)',
 'sepal width (cm)',
 'petal length (cm)',
 'petal width (cm)']
```

4D dataset → plotting two components

- **plotting the data**
- running k-means
- evaluating the result







see `Walk_Through_Kmeans.ipynb`

```
nClust      = 3
rep         = 25
dist        = distance_metric(type_metric.EUCLIDEAN)



my_model    = KMeansClusterer(nClust, distance = dist,\
                repeats   = rep,\
                avoid_empty_clusters = True)

PredLabels  = my_model.cluster(X2D,\
                assign_clusters = True)

Center      = my_model.means()
```

- plotting the data
- **running k-means**
- evaluating the result

we need to “guess” the number of cluster

the **initial** means are assigned randomly.
→ repeat the procedure 25 times
→ avoiding local minimum,

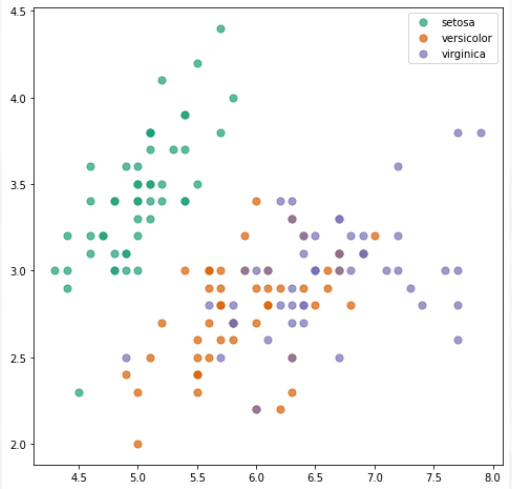
The features are meassured in cm, i. e. the correct distance to pick here is Euclidean

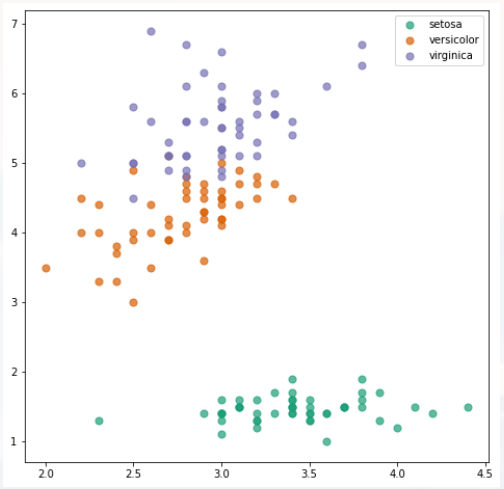
see `Walk_Through_Kmeans.ipynb`

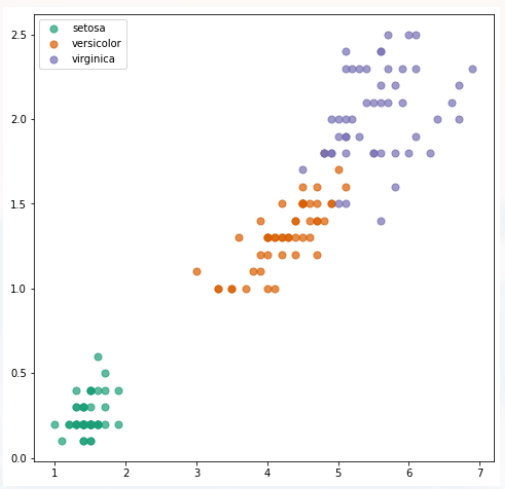
Clustering and Classification – Unsupervised

- plotting the data
- **running k-means**
- evaluating the result

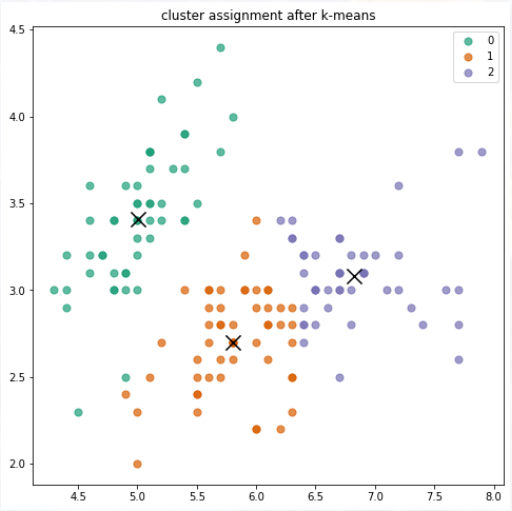
true classes

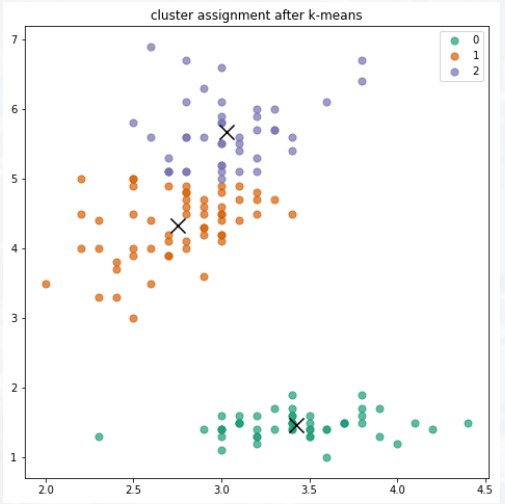


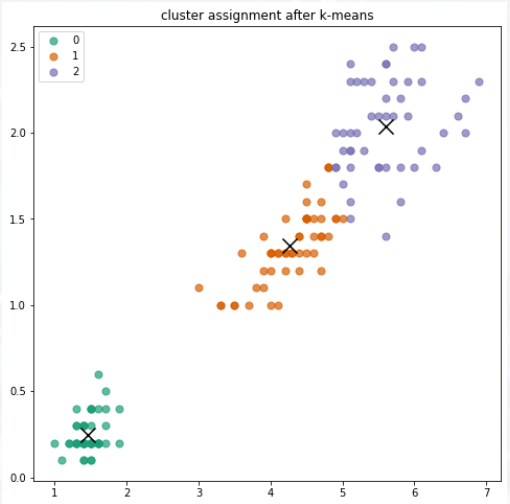




assigned classes





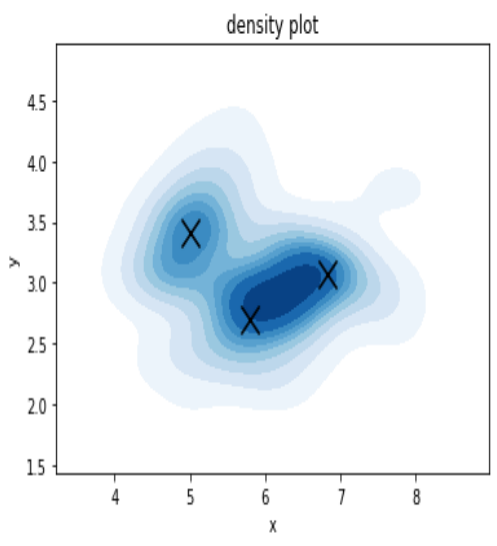


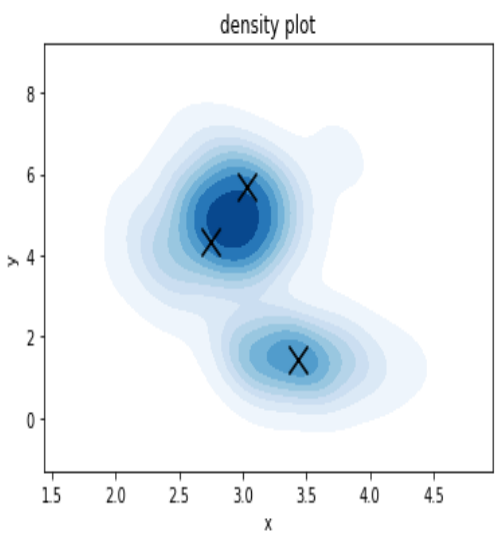
see `Walk_Through_Kmeans.ipynb`

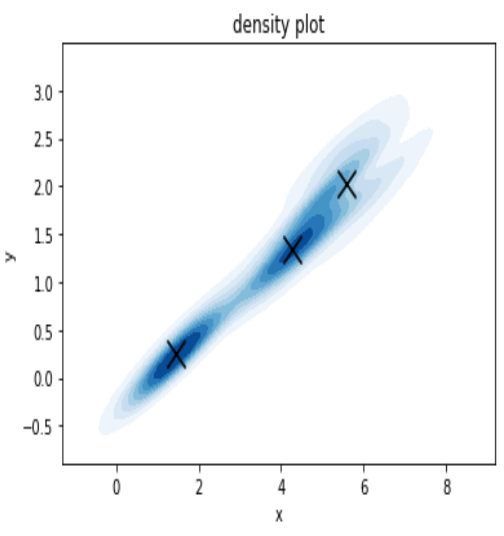
Clustering and Classification – Unsupervised

- plotting the data
- **running k-means**
- evaluating the result

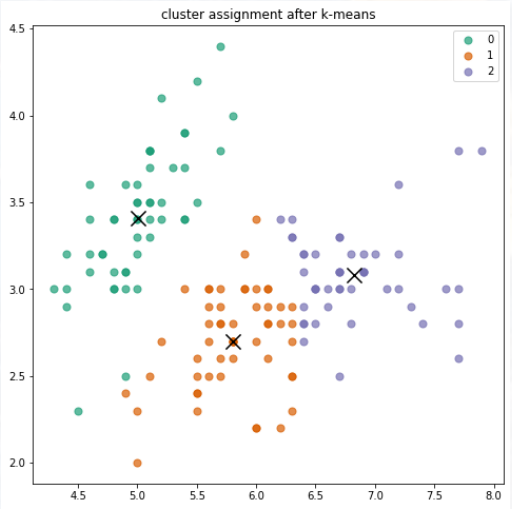
density

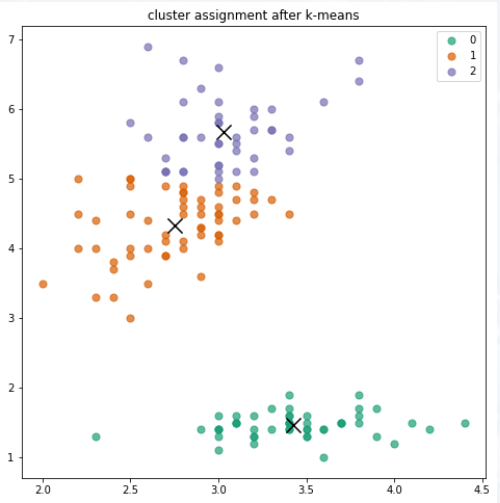


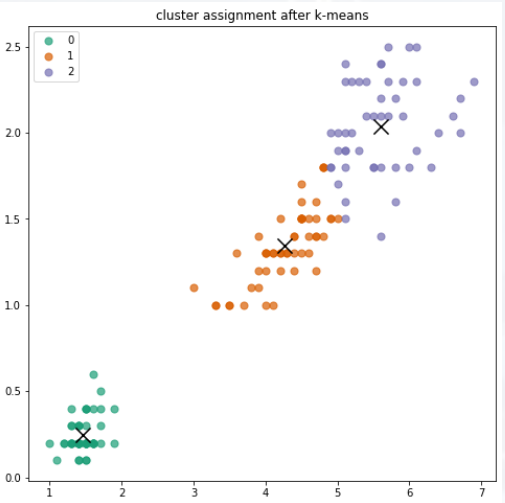




assigned classes







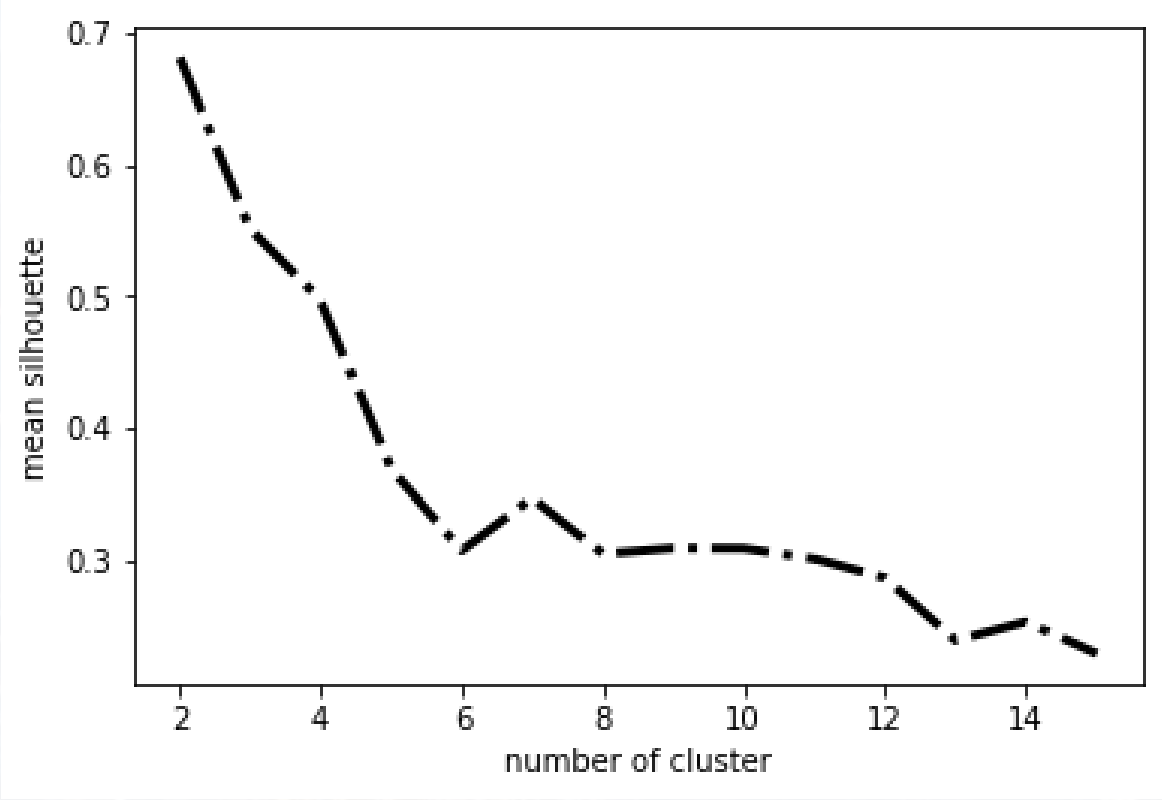
see `Walk_Through_Kmeans.ipynb`

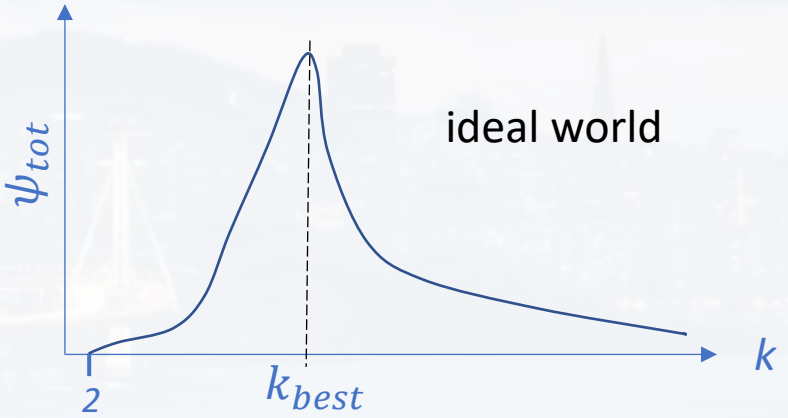
Clustering and Classification – Unsupervised

- plotting the data
- running k-means
- **evaluating the result**

we run k-means now for the **full 4D** dataset
+ evaluate clustering with silhouette

```
silhouette_score(X, Labels)
```





accuracy for 4D k = 3: 90%

see `Walk_Through_Kmeans.ipynb`

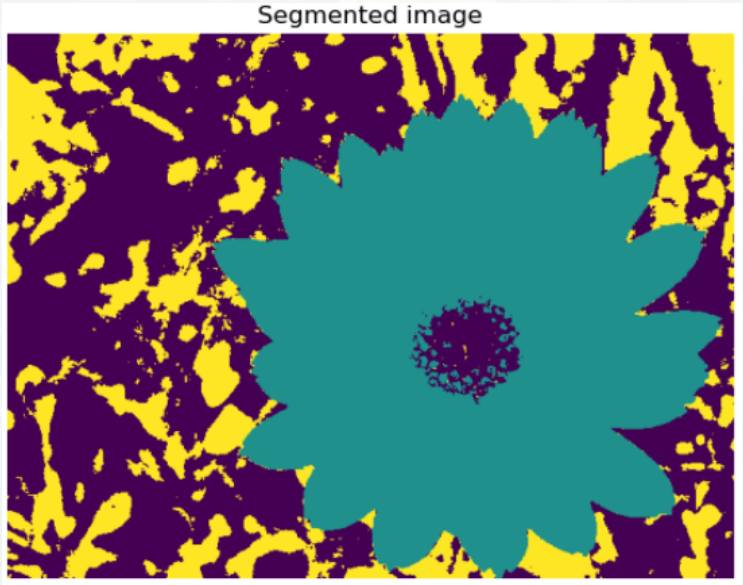
Clustering and Classification – Unsupervised

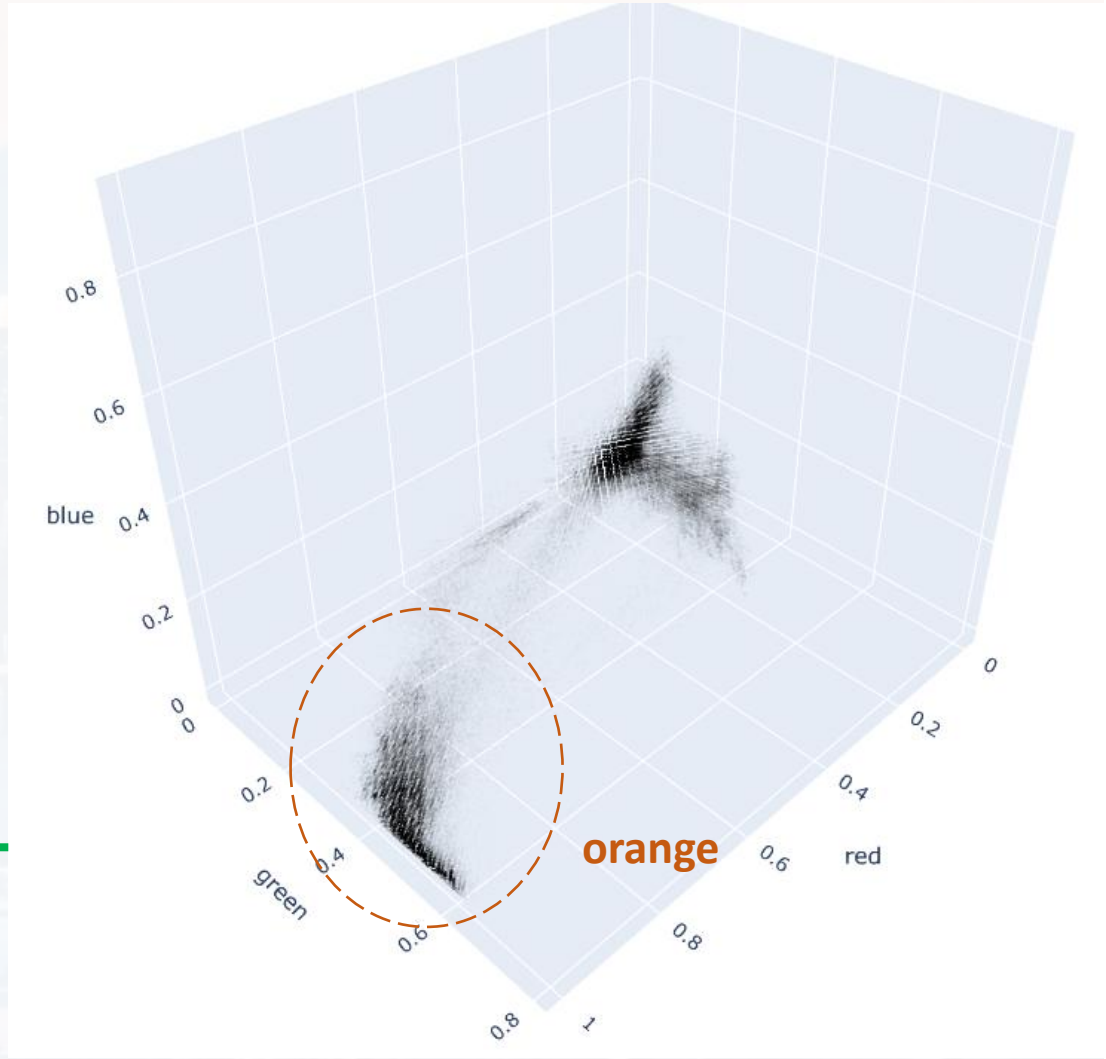
k-means for image processing



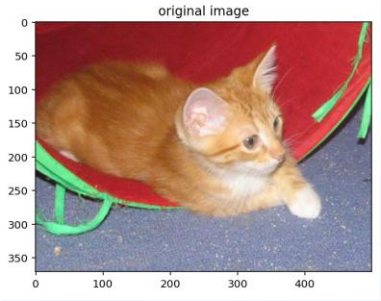
actual image

each pixel is a **vector in RGB** space





Clustering and Classification – Unsupervised

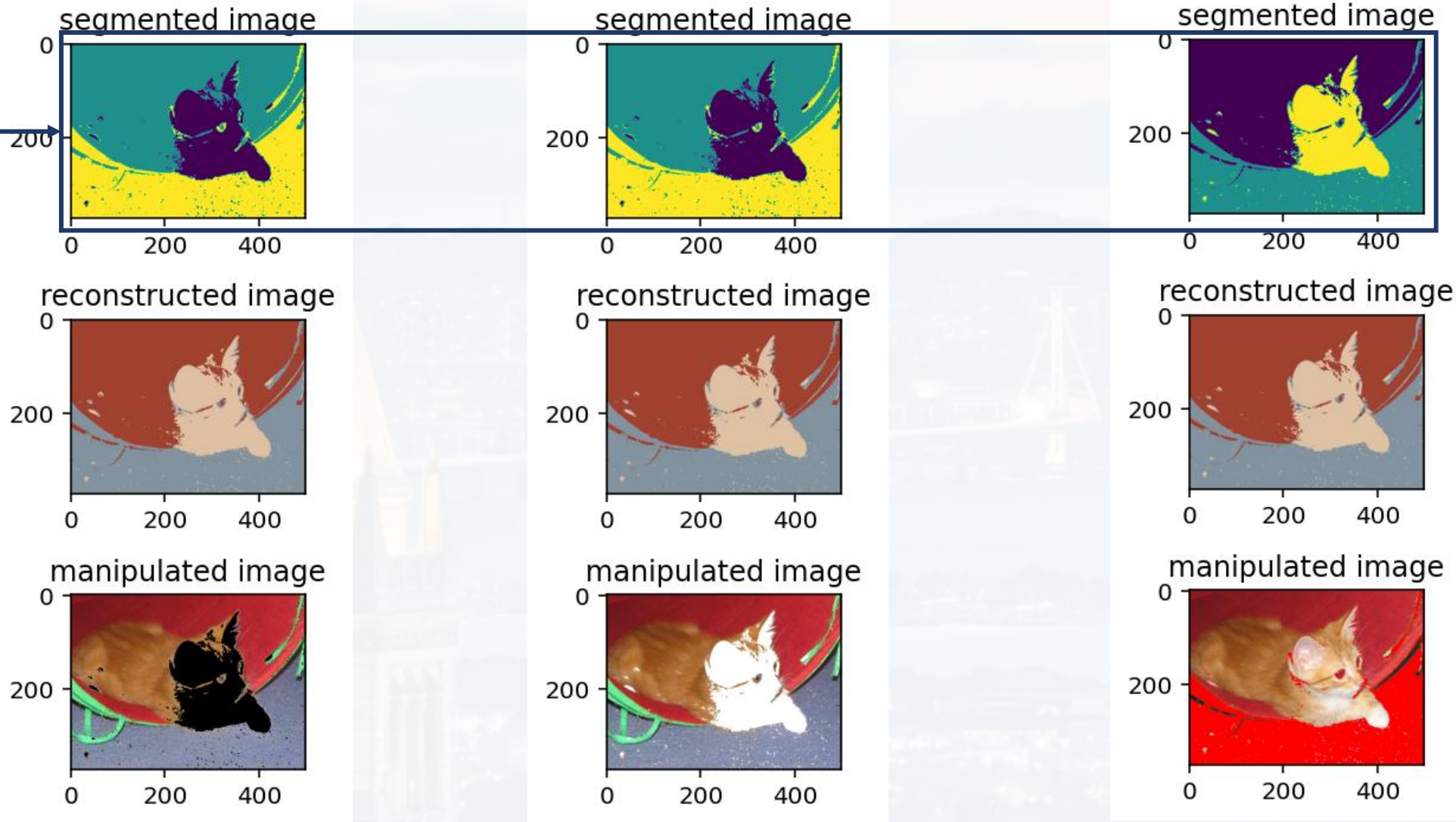


KMeansSgmentation.py

K.Segment()

K.Segment(values\
= [255, 255, 255])

K.Segment(values\
= [255, 0, 0], label = 1)



labels can swap **(k-means is unsupervised)**

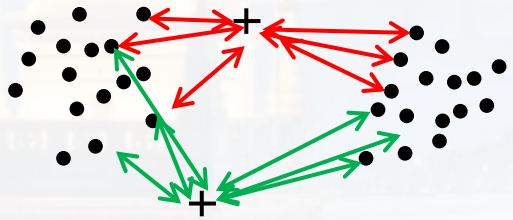
Clustering and Classification – Unsupervised

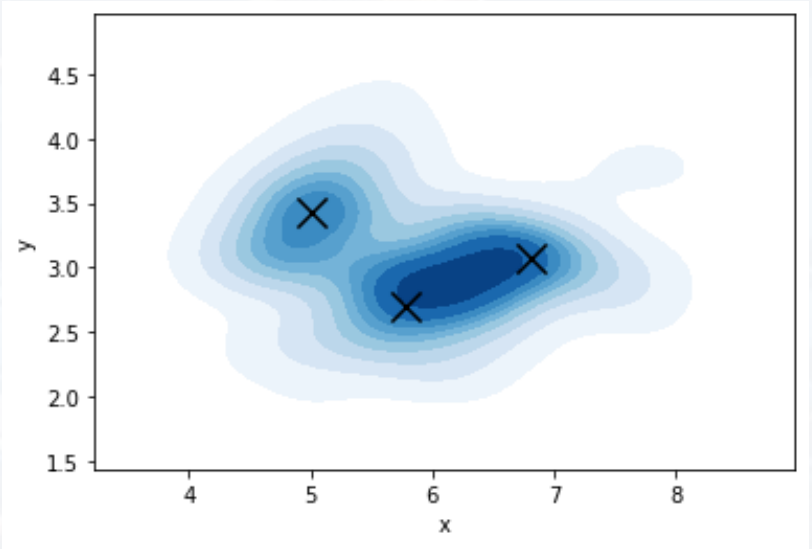
summary:

- simple and fast
- unsupervised
- *k* has to be given → silhouette for determining best *k*
- problems if cluster have unusual shapes

(elongated, hollow inside, scattered)

- **image segmentation!**

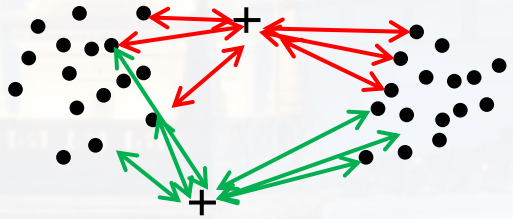


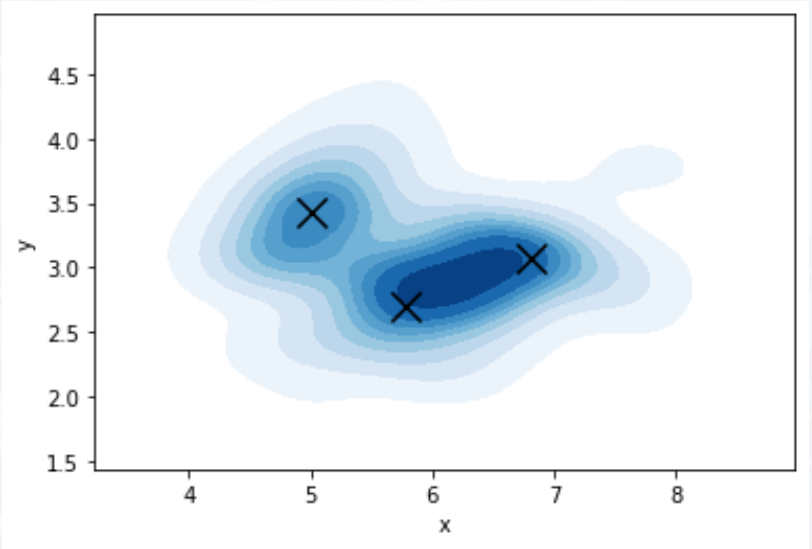


see `Walk_Through_Kmeans.ipynb`

topics for the discussion/office hour:

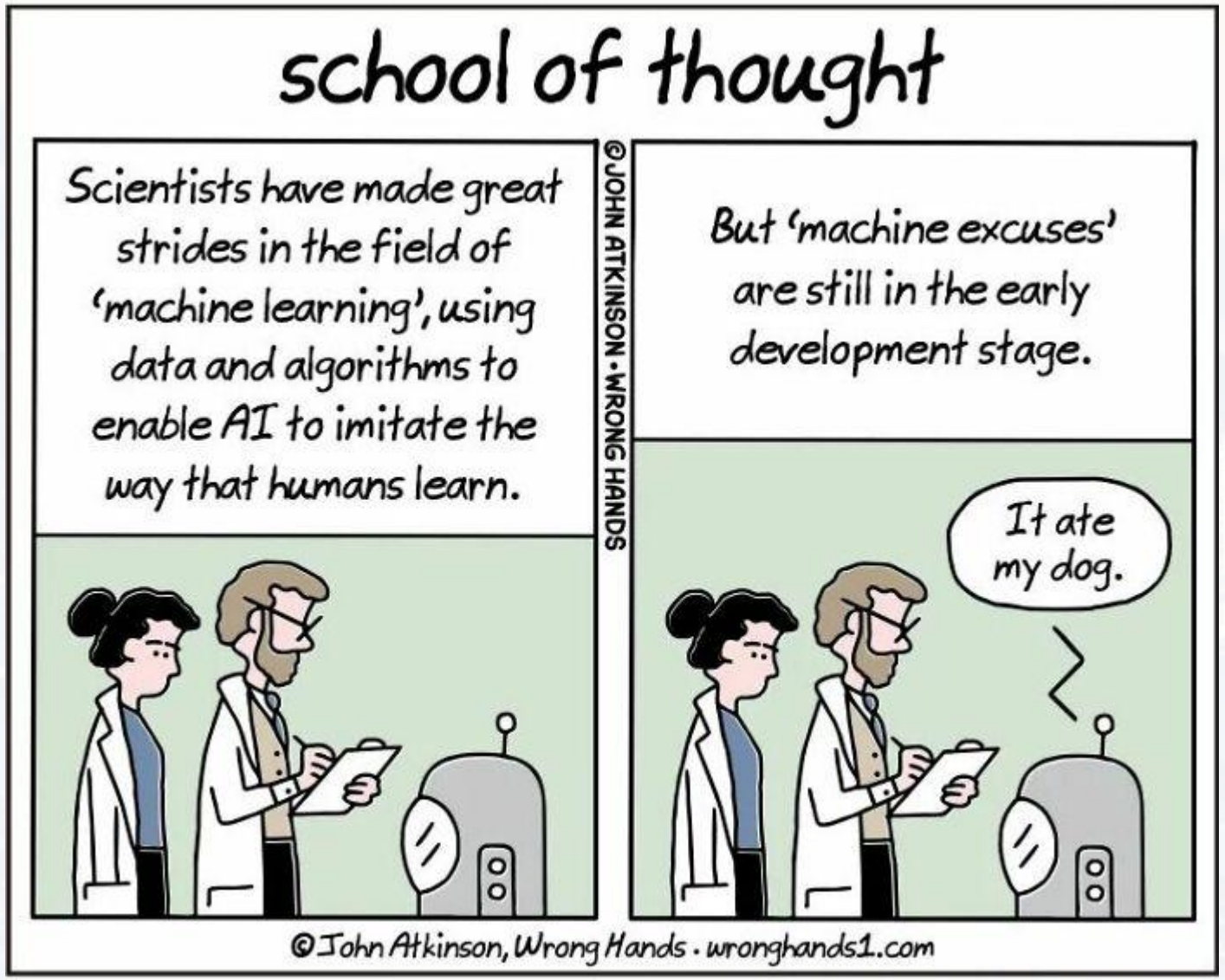
- What is a *distance*?
- Which are different distances?
- When to use which distance?





see `Walk_Through_Kmeans.ipynb`

Clustering and Classification - Unsupervised



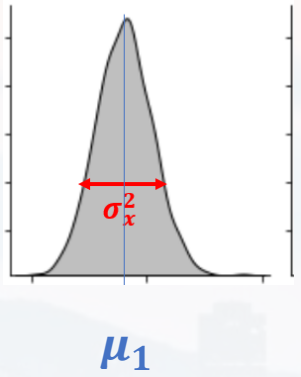
Outline

- Overview
- More Statistics
- **Examples**
  - k-means
  - GMMs
  - Trees

Gaussian **M**ixture **M**odels

one feature

$$N_1(x_1) = \frac{1}{\sqrt{2\pi\,\sigma_{x1}^2}} \exp\left\{-\frac{1}{2}\left(\frac{x_1 - \mu_1}{\sigma_{x1}}\right)^2\right\}$$

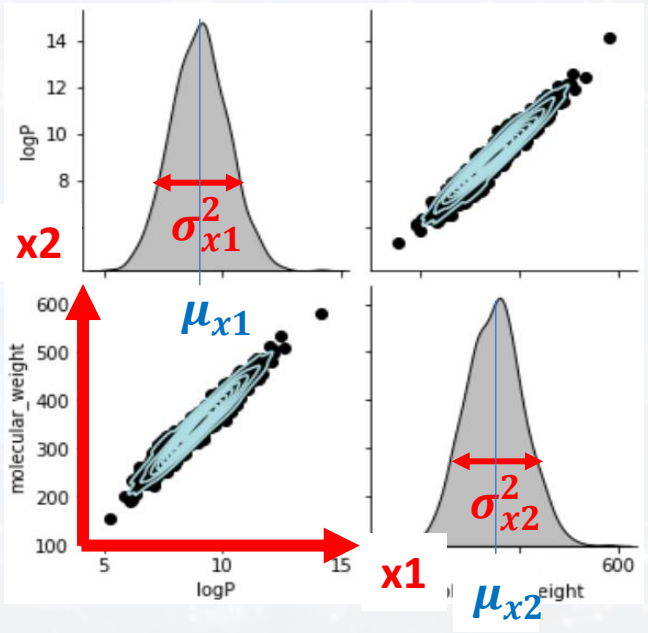


two features

$$\Sigma = \begin{pmatrix} \sigma_{x1}^2 & cov(x_1, x_2) \\ cov(x_2, x_1) & \sigma_{x2}^2 \end{pmatrix}$$ covariance matrix

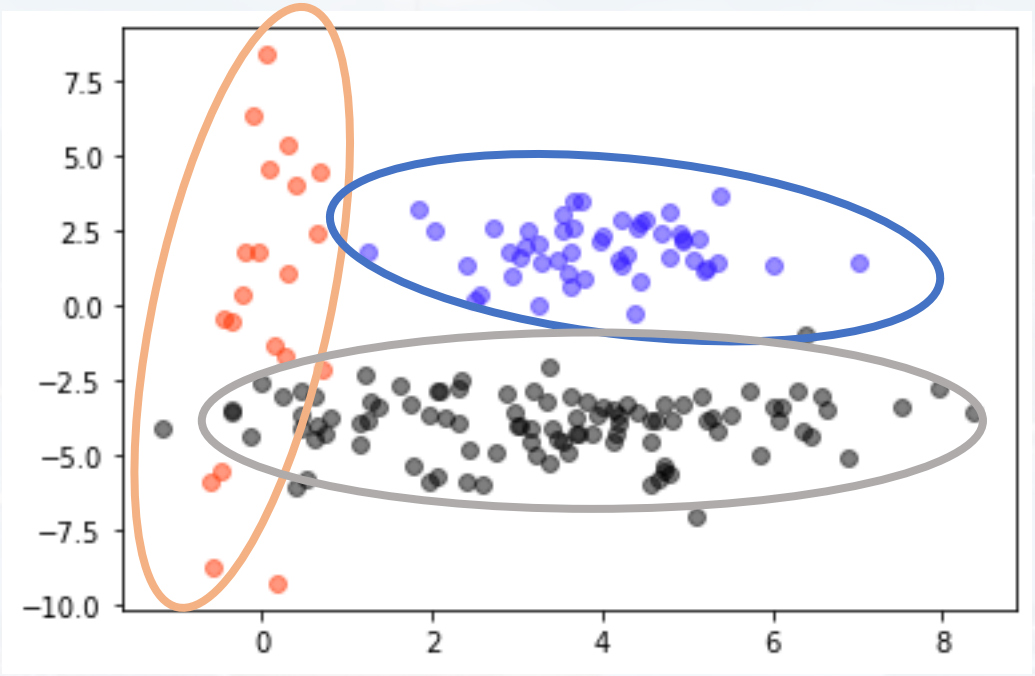
$$\begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix}^T \Sigma^{-1} \begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix}$$ see PCA lecture

$$N_2(x_1, x_2) = \frac{1}{2\pi\,\det(\Sigma)^{1/2}} \exp\left\{-\frac{1}{2}\left[\begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix}^T \Sigma^{-1} \begin{pmatrix} x_1 - \mu_{x1} \\ x_2 - \mu_{x2} \end{pmatrix}\right]\right\}$$

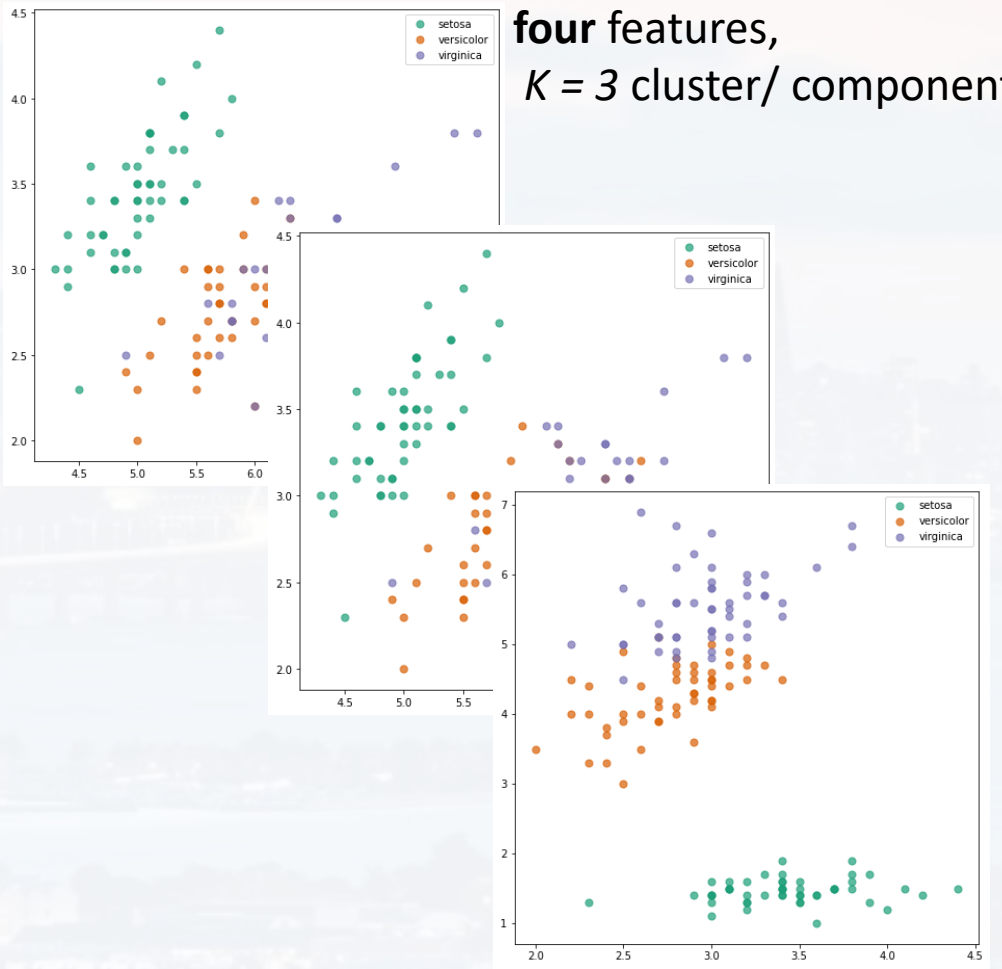


Gaussian **M**ixture **M**odels

two features, *K = 3* cluster/components



four features,
K = 3 cluster/ components



k-means

the actual problem:

observation x_n has been drawn from any of the cluster C_k

K : number of cluster
N : number of observations
$\boldsymbol{\mu_k}$: barycenter of the cluster

$$r_{n,k} = \begin{cases} \mathbf{1}, & \boldsymbol{if\ k = argmin_j \|x_n - \mu_j\|^2} \\ \mathbf{0}, & \boldsymbol{else} \end{cases}$$

marginalization

$$P(x_n) = \sum_{k=1}^{K} P(x_n|C_k)\, P(C_k) \qquad \sum_{k=1}^{K} P(C_k) = 1$$

$$J = \sum_{n=1}^{N} \sum_{k=1}^{K} r_{n,k}\ \|x_n - \mu_k\|^2$$

$P(x_n|C_k)$ likelihood function → describing the cluster density (Gaussian?)

$P(C_k)$ mixing coefficient $\boldsymbol{\pi_k}$ → proportional to size of the cluster

general:

$$P(x_n) = \sum_{z} P(x_n|z)\, P(z)$$

z: latent variable **(i. e. not observable, but can be inferred from $\{x_n\}$)**

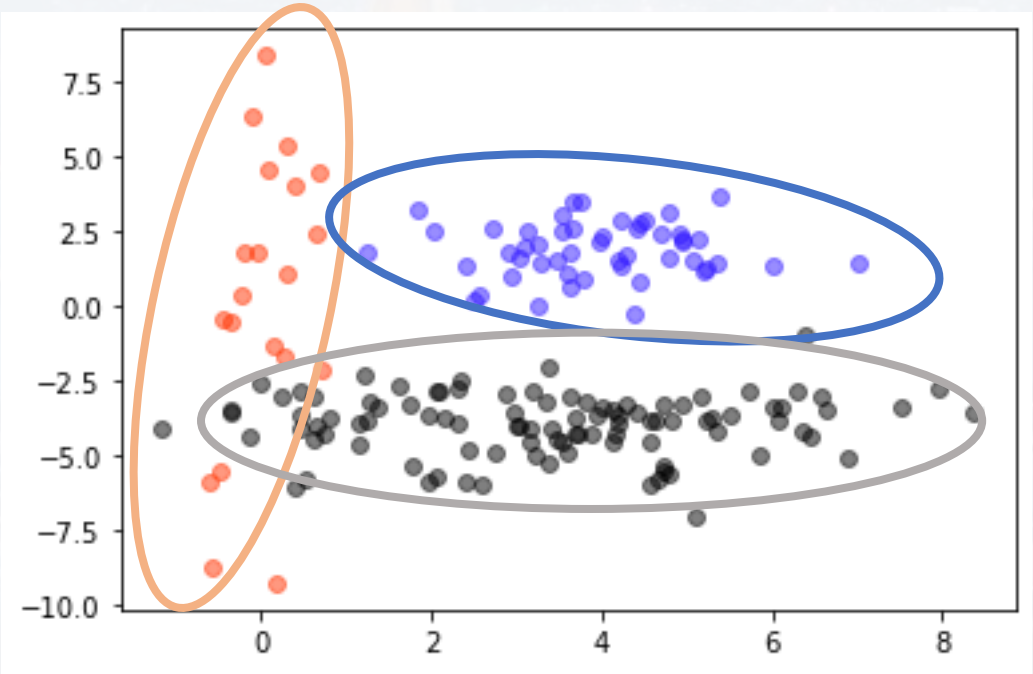
example: **G**aussian **M**ixture **M**odels

K	**: number of cluster**
π_k	**: mixing coefficient**

f features

$$\mathcal{N}_k(x|\mu_k,\Sigma_k) = \frac{1}{(2\pi)^{f/2}\det(\Sigma_k)^{1/2}}\, exp\left[-\frac{1}{2}(x-\mu_k)^T{\Sigma_k}^{-1}(x-\mu_k)\right]$$

two features: x is a vector $x = (x_1; x_2)$



two features, *K = 3* components

$$P(x) = \sum_z P(x|z)\,P(z)$$

$$= \sum_{k=1}^{K} \mathcal{N}_k(x|\mu_k,\Sigma_k)\,\pi_k$$

probability to pick a cluster C_k

example: **G**aussian **M**ixture **M**odels

N	**: number of observations**
K	**: number of cluster**
π_k	**: mixing coefficient**

marginalization $$P(x_n) = \sum_z P(x_n|z)\, P(z) = \sum_{k=1}^{K} \mathcal{N}_k(x_n|\mu_k, \Sigma_k)\, \pi_k$$

indicator variable $z_k \, \epsilon \, \{0, 1\}$

goal: $$P(z_k = 1|x_n) = \frac{P(z_k = 1)\, P(x_n|z_k = 1)}{P(x_n)} = \frac{\pi_k\, \mathcal{N}_k(x_n|\mu_k, \Sigma_k)}{\sum_{j=1}^{K} \mathcal{N}_j(x_n|\mu_j, \Sigma_j)\, \pi_j}$$

via maximizing likelihood L:

$$L = ln[P(x|\pi, \mu, \Sigma)] = ln\left[\prod_n P(x_n|\pi, \mu, \Sigma)\right] = \sum_{n=1}^{N} ln\left\{\sum_{k=1}^{K} \pi_k\, \mathcal{N}_k(x_n|\mu_k, \Sigma_k)\right\}$$

$$\boldsymbol{P(x_n)}$$

model parameter $\theta = \{\pi, \mu, \Sigma\}$

Gaussian **M**ixture **M**odels

idea: fitting the data to a GMM → analytical functions to **calculate** probabilities for labels

different algorithms:
- Bayesian
- **E**xpectation **M**aximization
- …

```
my_model   = GaussianMixture(n_components = k, random_state = 0).fit(X)

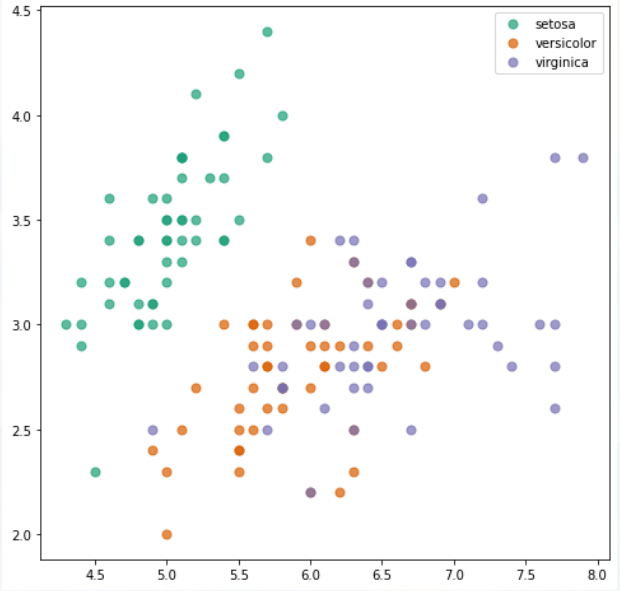
Center     = my_model.means_
PredLabels = my_model.predict(X)
```

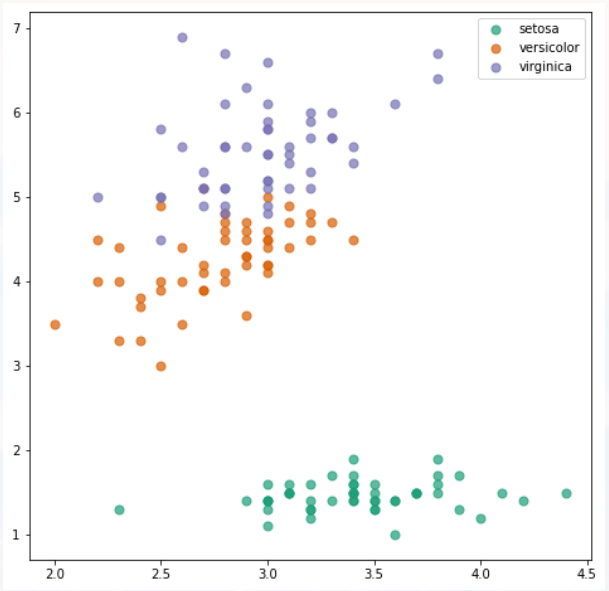
setting initial labels randomly

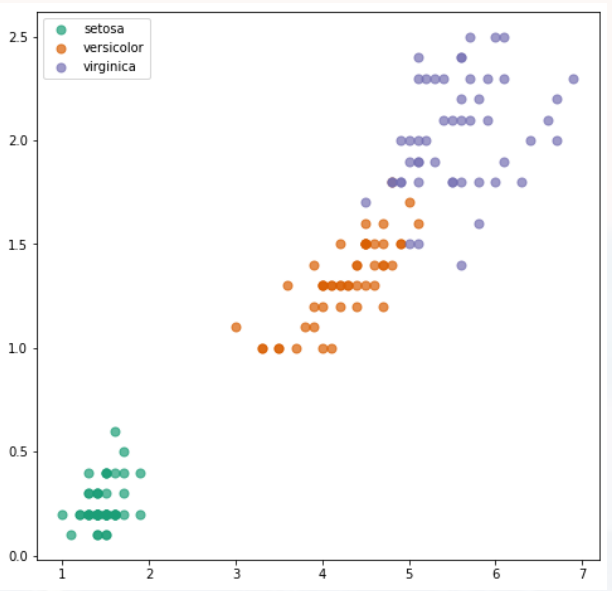
check out **Walk_Through_GMM.ipynb**

Clustering and Classification – Unsupervised

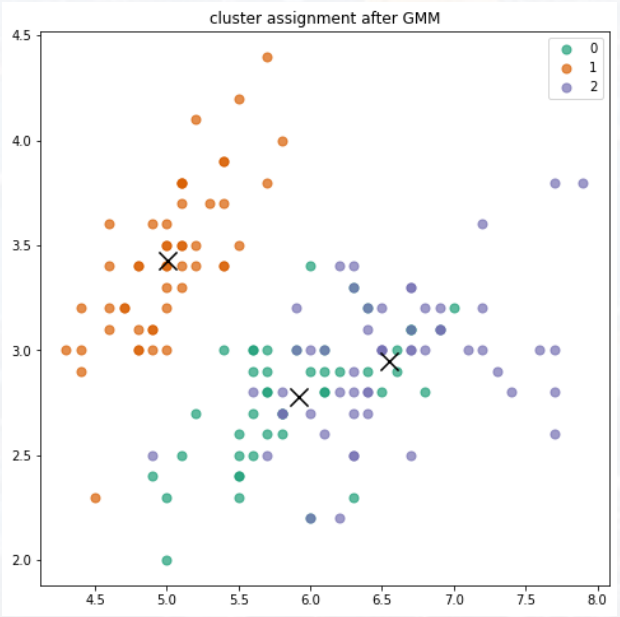
true classes

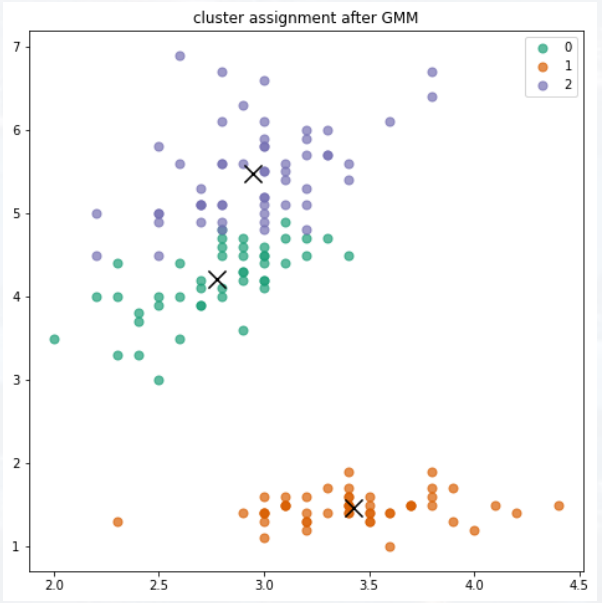


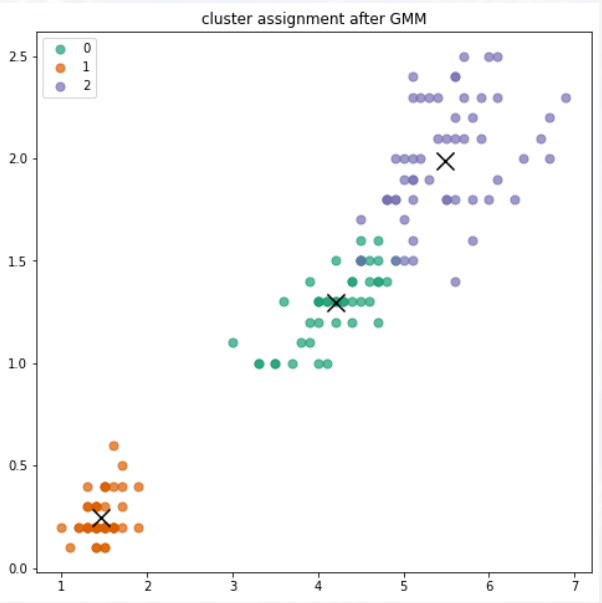




assigned classes

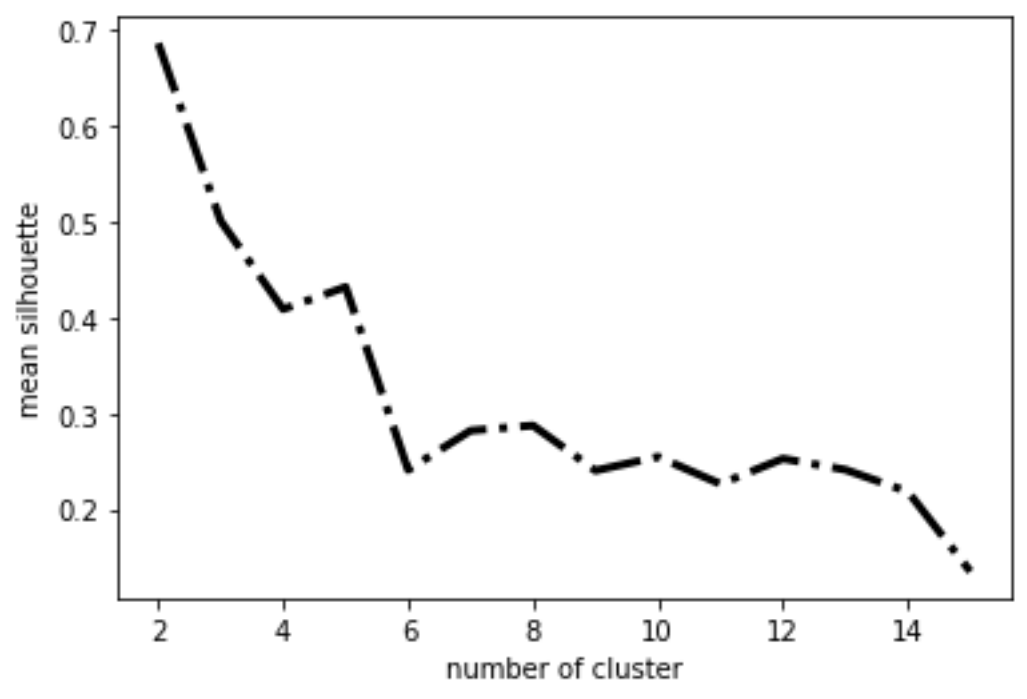






Clustering and Classification – Unsupervised

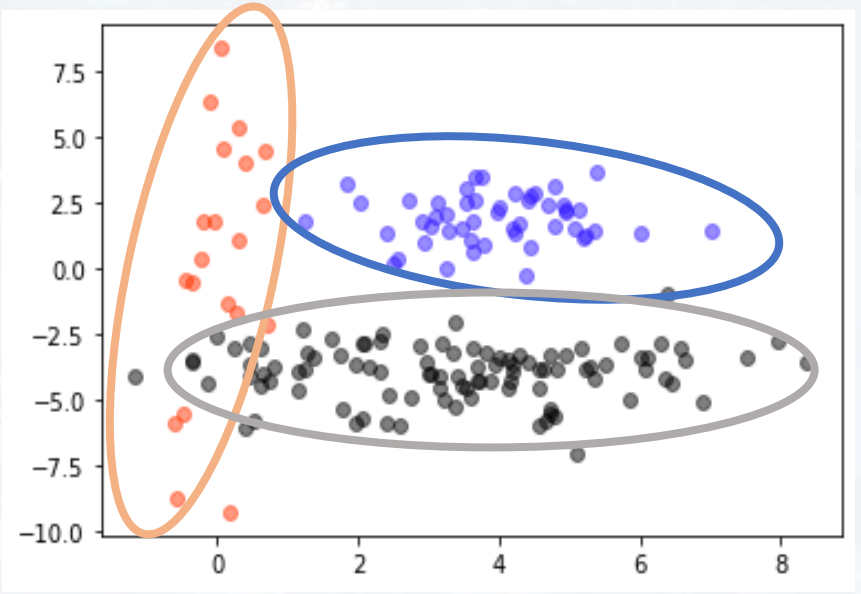
we run the GMM now for the **full 4D** dataset
+ evaluate clustering with silhouette



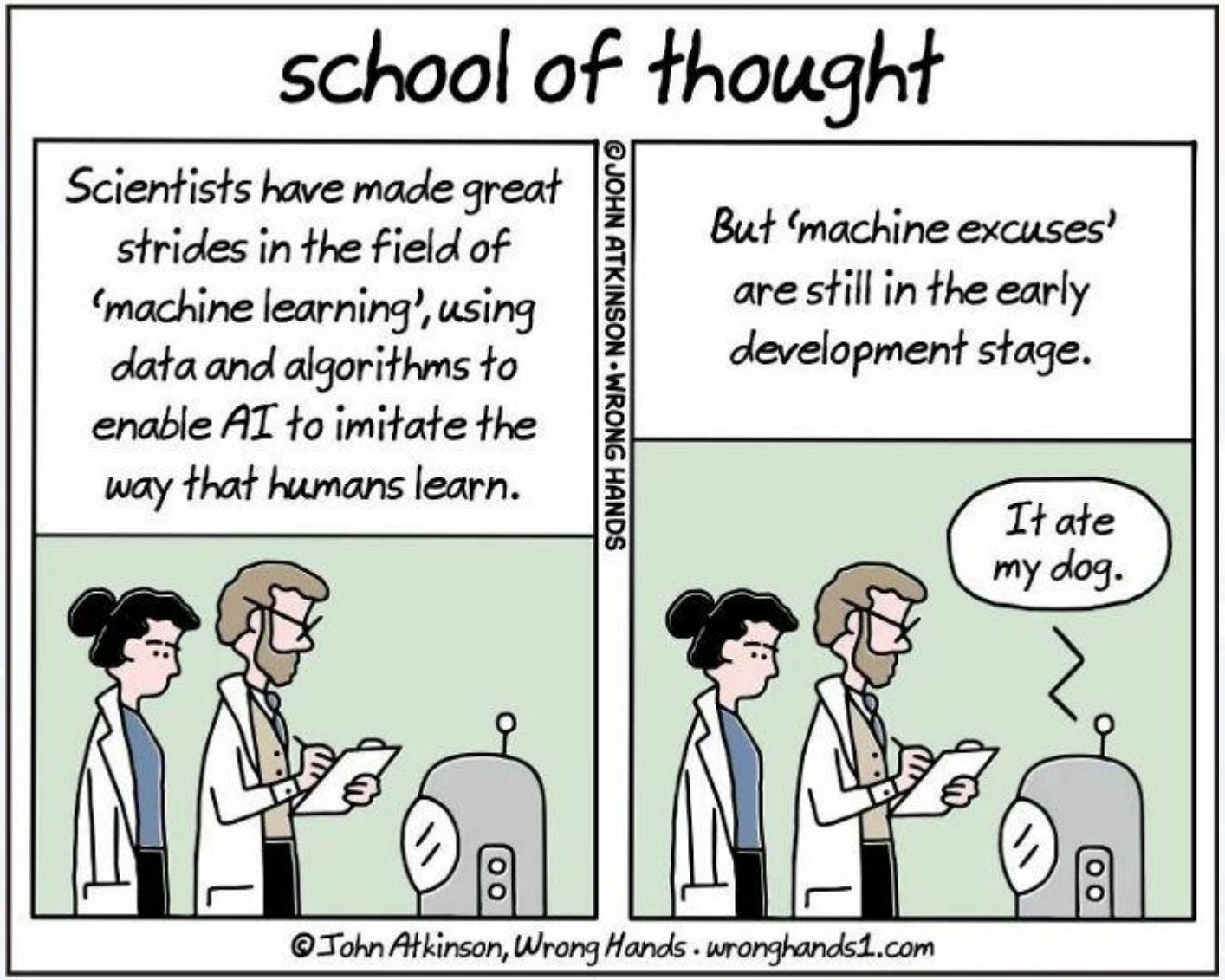
accuracy for 4D k = 3: 93%

topics for the discussion/office hour:

- EM algorithm
- mean, variance and covariance in more detail



Clustering and Classification - Unsupervised

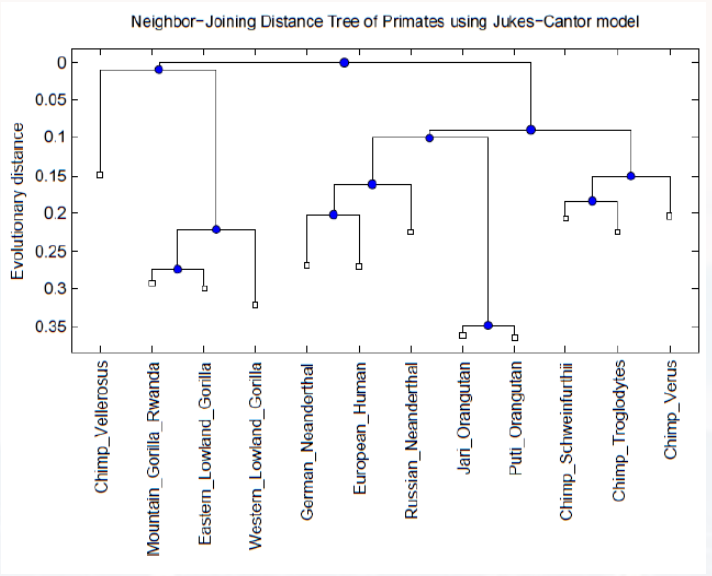


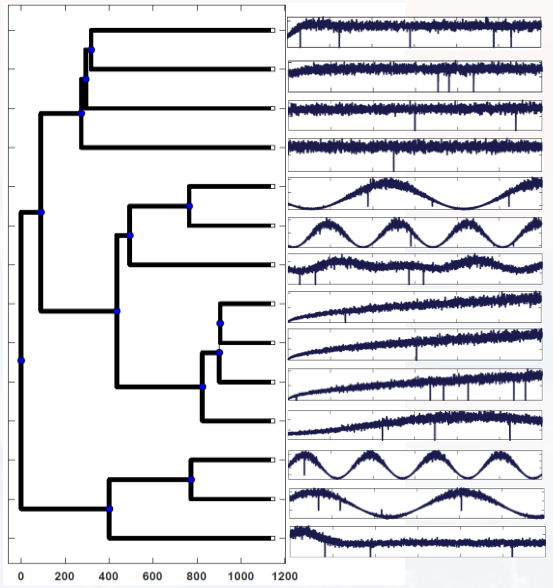
Outline

- Overview
- More Statistics
- **Examples**
  - k-means
  - GMMs
  - Trees

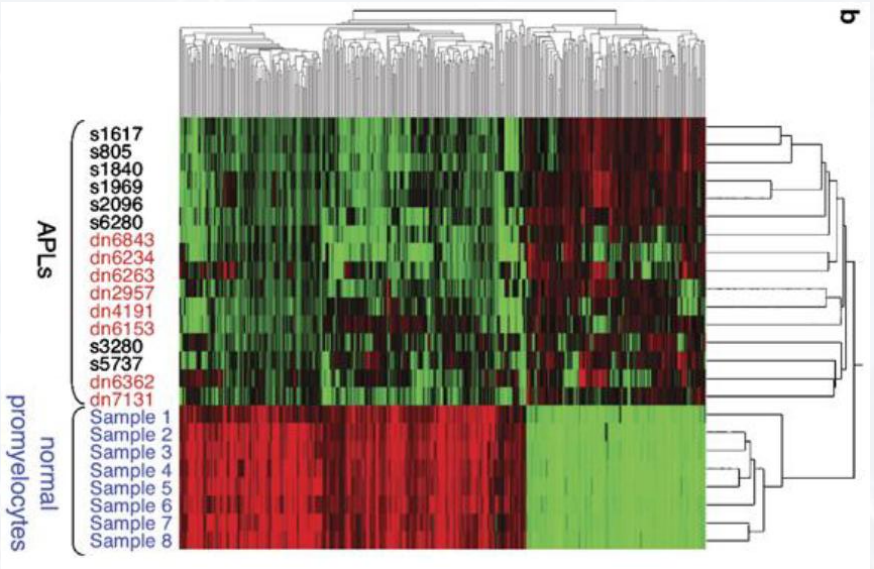
Clustering and Classification – Unsupervised





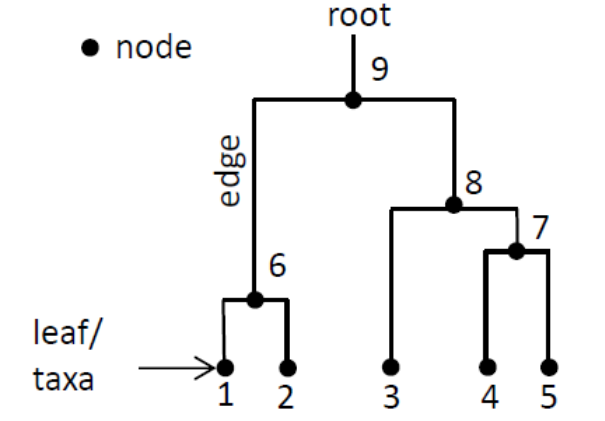


- What is a tree?
- Different kinds of trees...?
- How to build a tree?
- Why do we need trees?
- Examples...

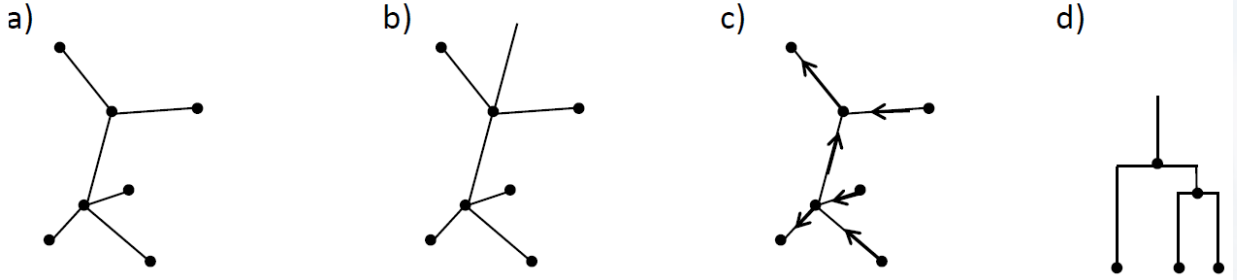


Clustering and Classification – Unsupervised

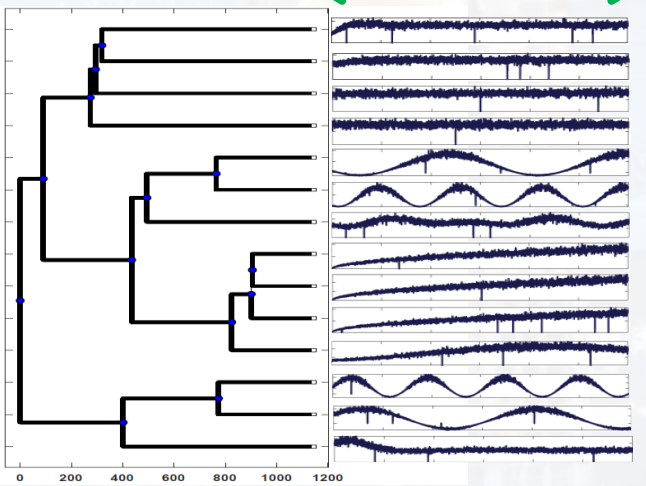
trees are a subclass of graphs **(but: not fully connected → "hierarchy", no loops)**:



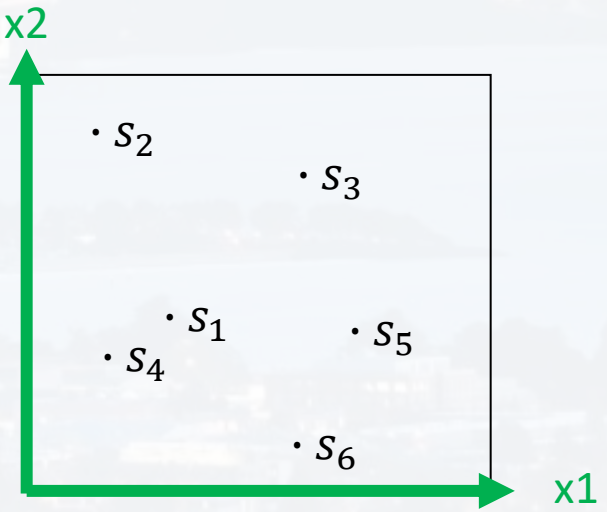
a) unrooted, undirected multinary tree
b) rooted, undirected multinary tree
c) unrooted, directed multinary tree
d) rooted, undirected binary tree



N timepoints

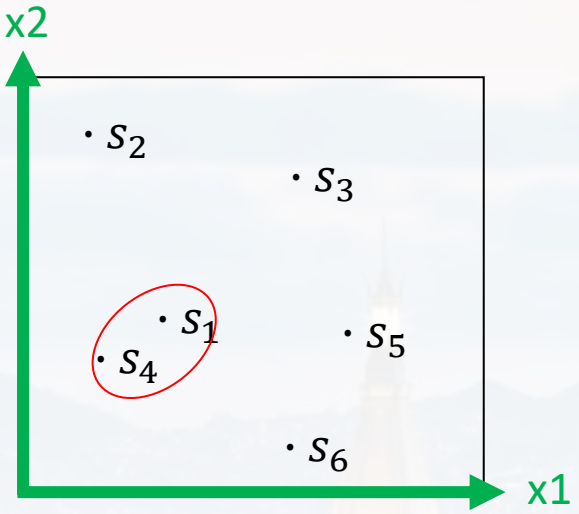


each sample s is a vector of N rows, hence, a data point in N-D



constructing trees:

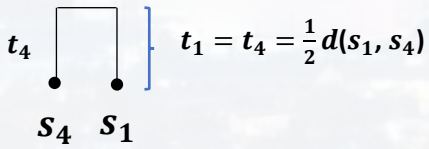
- calculating a distance between each pair of samples



	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0	$d(s_1, s_2)$	$d(s_1, s_3)$	$d(s_1, s_4)$	$d(s_1, s_5)$	$d(s_1, s_6)$
s_2		0	$d(s_2, s_3)$	$d(s_2, s_4)$	$d(s_2, s_5)$	$d(s_2, s_6)$
s_3			0	$d(s_3, s_4)$	$d(s_3, s_5)$	$d(s_3, s_6)$
s_4				0	$d(s_4, s_5)$	$d(s_4, s_6)$
s_5					0	$d(s_5, s_6)$
s_6						0

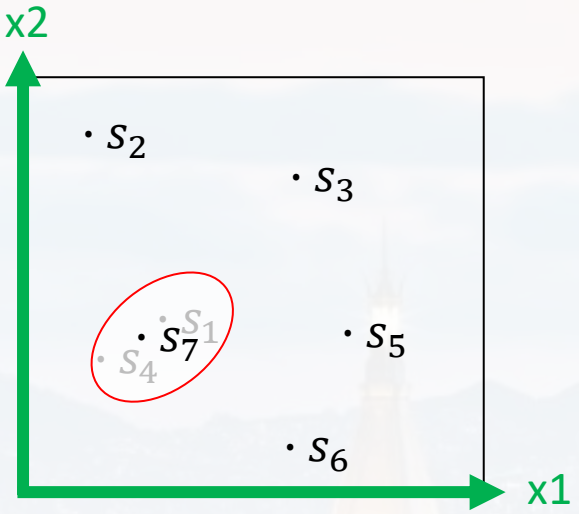
...once a distance has been defined (see soon)...

→ find the closest pair



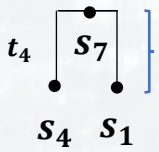
constructing trees:

- calculating a distance between each pair of samples



	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0	$d(s_1, s_2)$	$d(s_1, s_3)$	$d(s_1, s_4)$	$d(s_1, s_5)$	$d(s_1, s_6)$
s_2		0	$d(s_2, s_3)$	$d(s_2, s_4)$	$d(s_2, s_5)$	$d(s_2, s_6)$
s_3			0	$d(s_3, s_4)$	$d(s_3, s_5)$	$d(s_3, s_6)$
s_4				0	$d(s_4, s_5)$	$d(s_4, s_6)$
s_5					0	$d(s_5, s_6)$
s_6						0

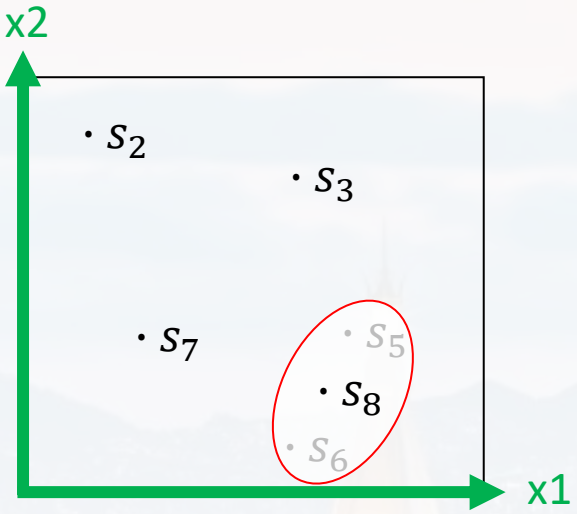
→ treat it as a new cluster $s_{1,4}$
→ use average of distance from cluster elements



$$t_1 = t_4 = \frac{1}{2} d(s_1, s_4)$$

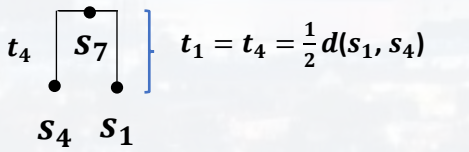
constructing trees:

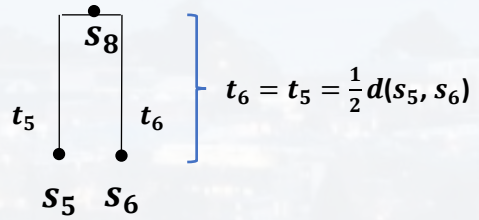
- calculating a distance between each pair of samples



	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0	$d(s_1, s_2)$	$d(s_1, s_3)$	$d(s_1, s_4)$	$d(s_1, s_5)$	$d(s_1, s_6)$
s_2		0	$d(s_2, s_3)$	$d(s_2, s_4)$	$d(s_2, s_5)$	$d(s_2, s_6)$
s_3			0	$d(s_3, s_4)$	$d(s_3, s_5)$	$d(s_3, s_6)$
s_4				0	$d(s_4, s_5)$	$d(s_4, s_6)$
s_5					0	$d(s_5, s_6)$
s_6						0

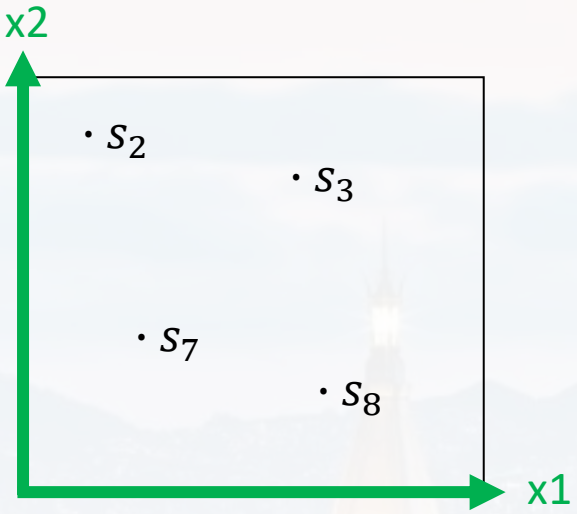
→ find the closest pair
(now including the cluster)





constructing trees:

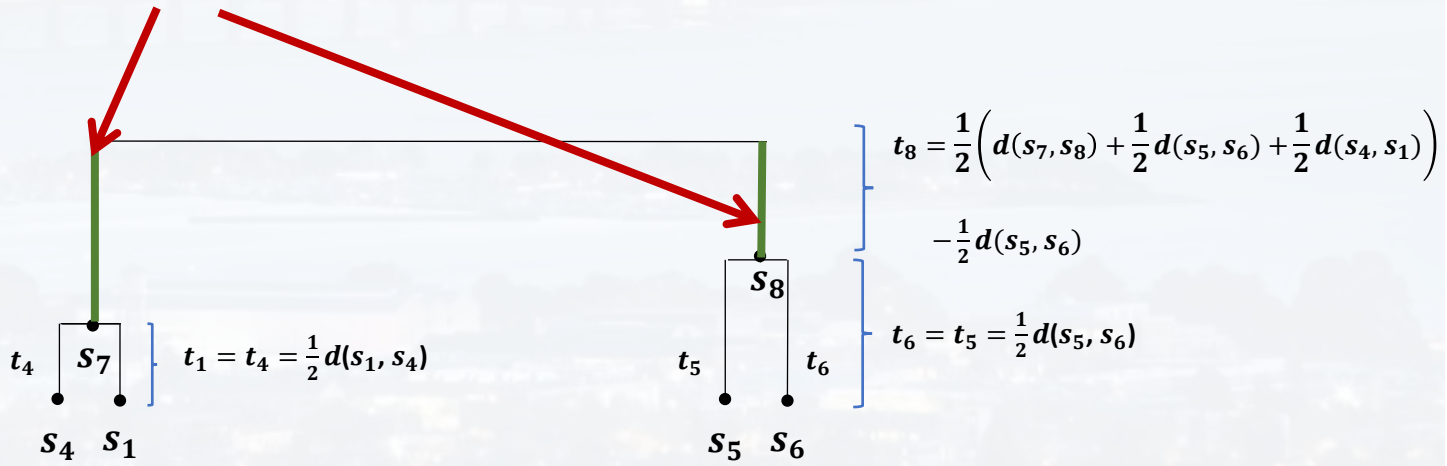
- calculating a distance between each pair of samples



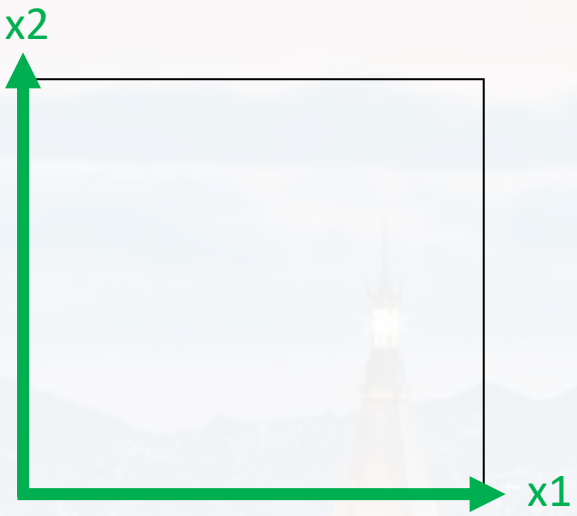
	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0	$d(s_1, s_2)$	$d(s_1, s_3)$	$d(s_1, s_4)$	$d(s_1, s_5)$	$d(s_1, s_6)$
s_2		0	$d(s_2, s_3)$	$d(s_2, s_4)$	$d(s_2, s_5)$	$d(s_2, s_6)$
s_3			0	$d(s_3, s_4)$	$d(s_3, s_5)$	$d(s_3, s_6)$
s_4				0	$d(s_4, s_5)$	$d(s_4, s_6)$
s_5					0	$d(s_5, s_6)$
s_6						0

→ and so on....

$$d(s_7, s_8) = \frac{d(s_5, s_7) + d(s_6, s_7)}{2} = \frac{d(s_1, s_5) + d(s_4, s_5) + d(s_1, s_6) + d(s_4, s_6)}{4}$$



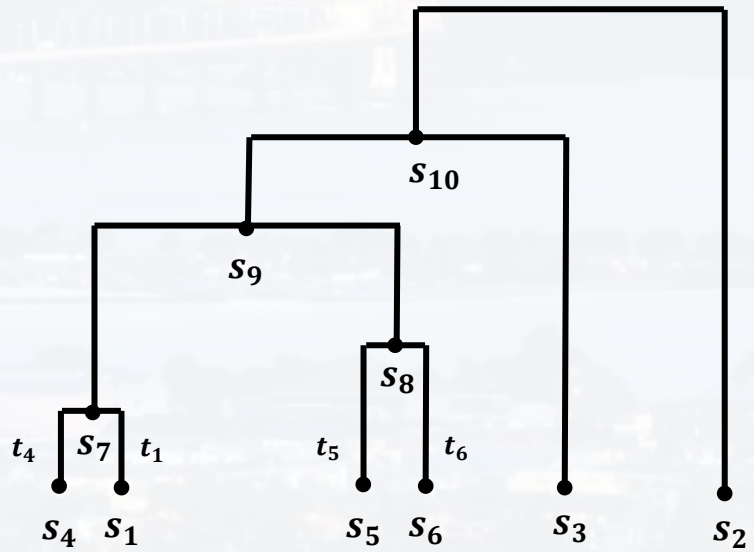
- calculating a distance between each pair of samples



	s_1	s_2	s_3	s_4	s_5	s_6
s_1	0	$d(s_1, s_2)$	$d(s_1, s_3)$	$d(s_1, s_4)$	$d(s_1, s_5)$	$d(s_1, s_6)$
s_2		0	$d(s_2, s_3)$	$d(s_2, s_4)$	$d(s_2, s_5)$	$d(s_2, s_6)$
s_3			0	$d(s_3, s_4)$	$d(s_3, s_5)$	$d(s_3, s_6)$
s_4				0	$d(s_4, s_5)$	$d(s_4, s_6)$
s_5					0	$d(s_5, s_6)$
s_6						0

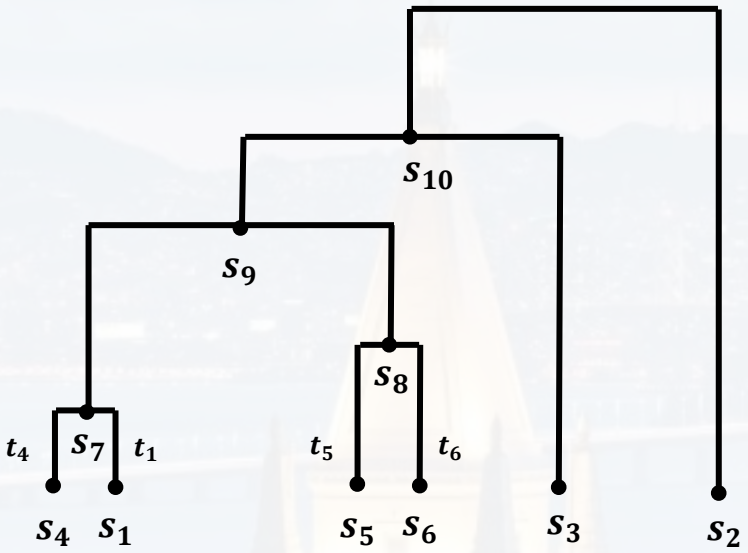
....finally

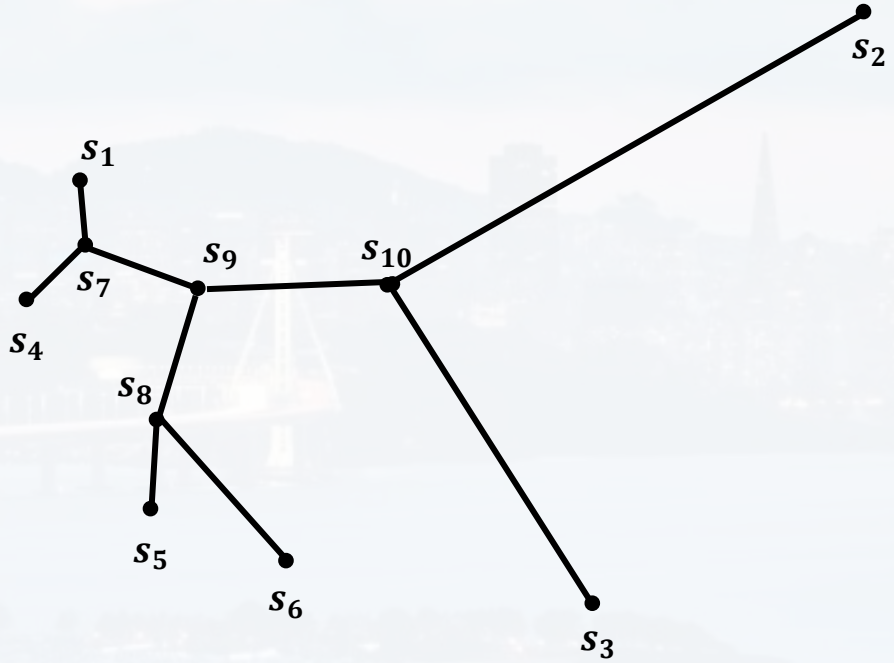
→ **U**nweighted **P**air **G**roup **M**ethod Using **A**rithmetic **A**verages (UPGMA)



constructing trees:

→ **U**nweighted **P**air **G**roup **M**ethod
Using **A**rithmetic **A**verages (UPGMA)





note: sometimes these diagrams might be misleading when interpreting the distances between the nodes

distances:

"A distance ***d*** is a function that assigns a **positive real number** to a set and/or to elements of a set."

properties:

- **d(x , y) ≥ 0**
- **d(x , y) = 0 ⇔ $x = y$**
- **d(x , y) = d(y , x)**
- **d(x , z) ≤ d(x , y) + d(y , z)**

frequently used distances:

- Euclidean
- cityblock
- cosine
- correlation
- hamming
- Jukes Cantor

distances:

"A distance ***d*** is a function that assigns a **positive real number** to a set and/or to elements of a set."

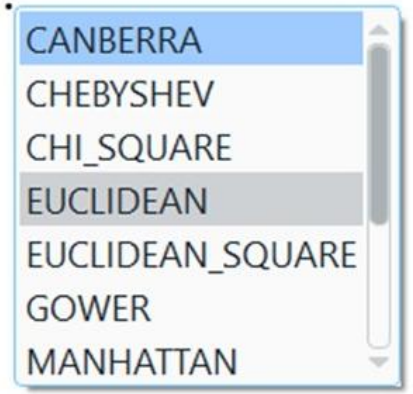
properties:

- **d(x , y) ≥ 0**
- **d(x , y) = 0 ⇔ $x = y$**
- **d(x , y) = d(y , x)**
- **d(x , z) ≤ d(x , y) + d(y , z)**

- Euclidean
- cityblock
- cosine
- correlation
- hamming
- Jukes Cantor

```
from pyclustering.utils.metric import *
```

```
type_metric.
```



distances:

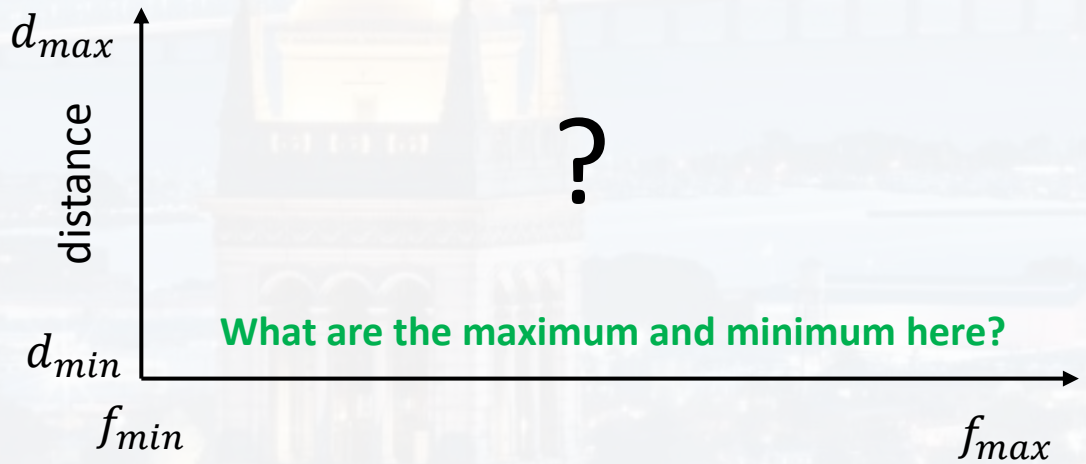
The Jukes-Cantor distance

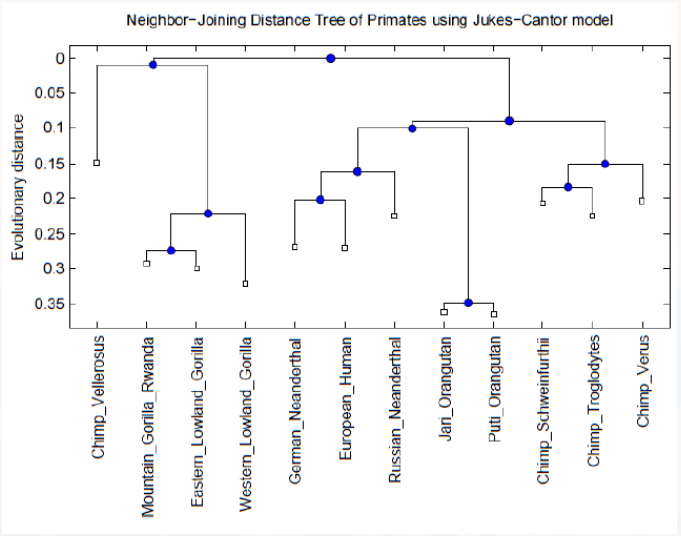
question: What could be a measure of distance here?

Seq1 = {AAGTCTTTGAATCGTATGGGCGCGGCTTTCAAAA}
Seq2 = {AGGTCTATGAATCGTATGGGTTCGGCTTTCAAAA}

fraction ***f*** of different sites

if ***f*** is large → ***d*** is large
if ***f*** is small → ***d*** is small

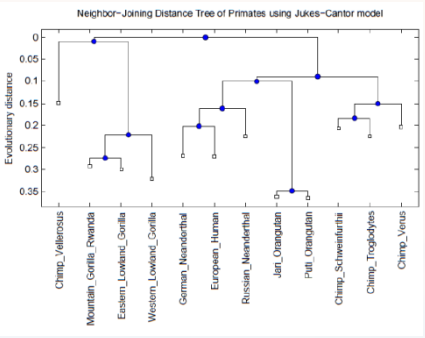




distances:

f : fraction of different sites

Seq1 = {AAGTCTTTGAATCGTATGGGCGCGGCTTTCAAAA}
Seq2 = {AGGTCTATGAATCGTATGGGTTCGGCTTTCAAAA}



The Jukes-Cantor distance

$f_{min} = 0 \rightarrow d_{min} = 0$ (sequences are identical)

for ***N*** letters (***N = 4*** nucleotides, ***N = 20*** amino acids)

$$f_{max} = 1 - \frac{1}{N} \rightarrow d_{\max} = \infty$$

(sequences have no relations, matches are coincidence)

- the probability that a site **has been** mutated, given a mutation rate λ equals:

$$P(any|\lambda) = 1 - e^{-\lambda t}$$

- the probability that this mutation will lead to **a particular letter** is

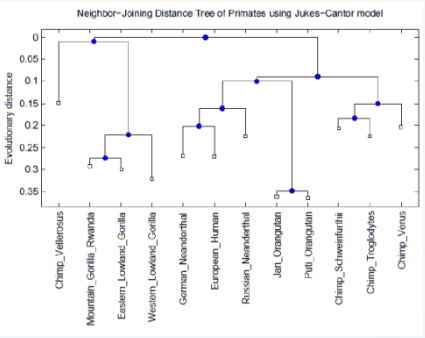
$$P(part|\lambda) = \frac{1}{N}(1 - e^{-\lambda t})$$

- the probability that this mutation will lead to a **letter other than the previous one** is

$$P(part|\lambda) = \frac{N-1}{N}(1 - e^{-\lambda t})$$

distances:

f : fraction of different sites



The Jukes-Cantor distance

Seq1 = {AAGTCTTTGAATCGTATGGGCGCGGCTTTCAAAA}
Seq2 = {AGGTCTATGAATCGTATGGGTTCGGCTTTCAAAA}

$f_{min} = 0 \rightarrow d_{min} = 0$ (sequences are identical)

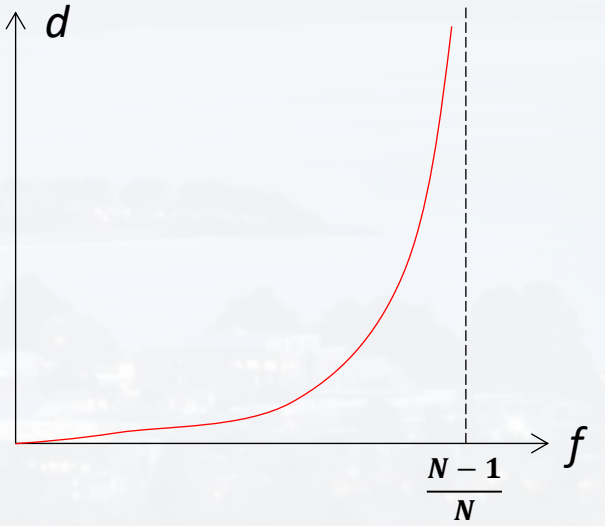
for ***N*** letters (***N = 4*** nucleotides, ***N = 20*** amino acids)

- the probability that this mutation will lead to a **letter other than the previous one** is

$$P(part|\lambda) = \frac{N-1}{N}(1 - e^{-\lambda t})$$

$\lambda t \sim d$ since cumulative mutations is what makes species different!

$$d \sim \lambda t = -ln\left(1 - \frac{N}{N-1} f\right)$$



often:

$$d = -\frac{N-1}{N} ln\left(1 - \frac{N}{N-1} f\right)$$

Jukes – Cantor distance

Clustering and Classification – Unsupervised

Jukes – Cantor distance $d(x,\bar{x}) = -\frac{N-1}{N} ln\left(1 - \frac{N}{N-1} f(x,\bar{x})\right)$

Euclidean distance $d(x,\bar{x})^2 = (x_1 - \bar{x}_1)^2 + (x_2 - \bar{x}_2)^2 \dots (x_N - \bar{x}_N)^2$

cityblock $d(x,\bar{x}) = |x_1 - \bar{x}_1| + |x_2 - \bar{x}_2| \dots |x_N - \bar{x}_N|$

cosine $d(x,\bar{x}) = 1 - \frac{x \cdot \bar{x}}{\sqrt{(x \cdot x)(\bar{x} \cdot \bar{x})}}$

correlation $d(x,\bar{x}) = 1 - \frac{cov(x,\bar{x})}{\sqrt{var(x)var(\bar{x})}}$

...and many others

note: for sequence alignment and phylogenetic trees, chemical properties of the AA or NT have to be taken into account → **score matrices**

see **`Walk_Through_Tree.ipynb`**

Python libraries:

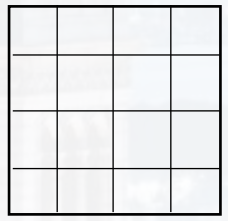
libraries from the *Bio* package

```
from Bio.Phylo.TreeConstruction import DistanceCalculator, DistanceTreeConstructor
from Bio import Phylo
from scipy import spatial
```

for plotting a phylogenetic tree

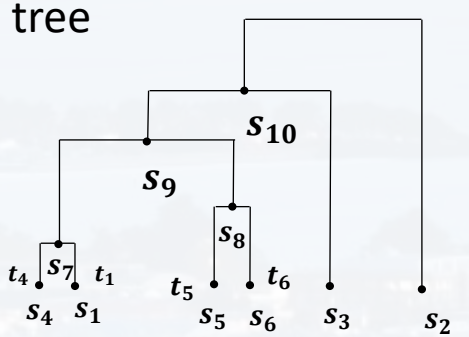
for rearranging a distance matrix

general idea: distance matrix



linkage algorithm (e.g. UPGMA)

tree



see **`Walk_Through_Tree.ipynb`**

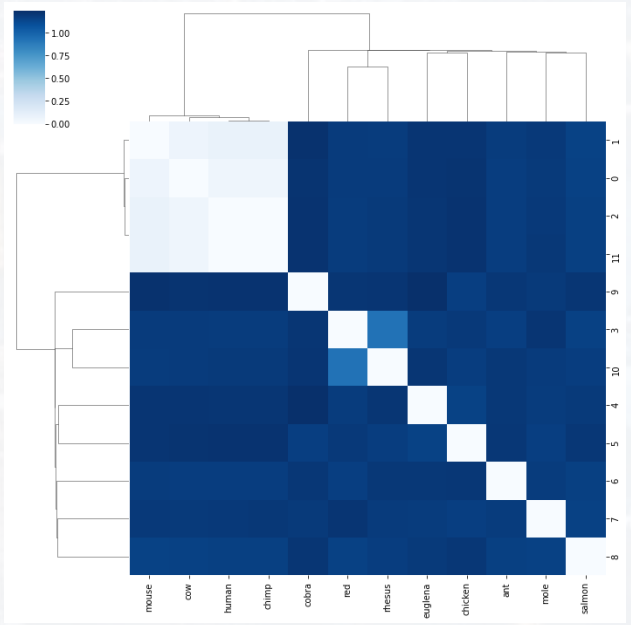
Python libraries:

also most heatmap tools have some abilities to construct trees: `sns.clustermap`

```
sns.clustermap(D_df, cmap = 'Blues', \
                     row_cluster = True, col_cluster = True, \
                     metric = 'euclidean', method = 'average', \
                     yticklabels = True)

plt.show()
```

similar to UPGMA



Clustering and Classification - Unsupervised

Thank you very much for your attention!

